



DIPLOMATERVEZÉSI FELADAT

Pásztori Péter Benjámín
Mérnök-informatikus hallgató részére

Egészséges életmódot támogató webalkalmazás fejlesztése ASP.NET MVC és Angular alapokon

Az egészséges életmód fenntartása és a táplálkozással kapcsolatos adatok nyomon követése napjainkban egyre nagyobb kihívást jelent. A tudatos étkezés és a rendszeres testmozgás összehangolása, valamint az ehhez szükséges információk rögzítése és visszakeresése gyakran bonyolult vagy időkieszt jelent. A modern webfejlesztési környezetek – mint például az ASP.NET MVC és az Angular – olyan lehetőségeket kínálnak, amelyekkel korszerű, felhasználóbarát, alkalmazások hozhatók létre. Az így elkészült szoftver segítséget nyújthat a mindennapi étkezések, receptek, tápanyagok és mozgásformák rugalmas vezetésében, és megfelelő elemzési lehetőségeket kínálhat az életmód célok eléréséhez (például fogyás vagy tömegnövelés). A hallgató feladata egy olyan webalkalmazás megtervezése és megvalósítása, amely a regisztrált felhasználók életmódjának nyomon követését és támogatását szolgálja.

A hallgató feladatának a következőkre kell kiterjednie:

- Felhasználókezelés és profilbeállítás
- Alapanyag- és receptkezelés:
 - Alapanyag rögzítése tápanyag-információkkal (fehérje, zsír, szénhidrát, kalória).
 - Receptek létrehozása és szerkesztése, amelyek alapanyagokból épülnek fel.
- Az elfogyasztott ételek rögzítése a naplóba, és a kalória-/tápanyagtartalom összesítése.
- Sporttevékenységek, mozgásformák felvétele.
- Célkövetés és visszajelzés: Az aktuális napi tápanyagbevitel (kalória, makrotápanyagok) folyamatos összevetése a profilban beállított céllal, vizuális visszajelzéssel.
- Előzmények és statisztikák:
 - Korábbi napok adatai (étkezések, mozgás).
 - A testsúly alakulásának grafikus megjelenítése.

Tanszéki konzulens: Dr. Kővári Bence, egyetemi docens

Budapest, 2025. február 26.

Dr. Charaf Hassan
egyetemi tanár
tanszékvezető





Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

Pásztori Péter Benjámin

EGÉSZSÉGES ÉLETMÓDOT TÁMOGATÓ WEBALKALMAZÁS FEJLESZTÉSE ASP.NET MVC ÉS ANGULAR ALAPOKON

KONZULENS

Dr. Kővári Bence András

BUDAPEST, 2025

Tartalomjegyzék

Összefoglaló	6
Abstract.....	7
1 Bevezetés	8
1.1 A digitális egészségtudatosság kora	8
1.2 A probléma felvetése és a fejlesztésem célja.....	9
1.3 A megoldás szakmai motivációja	9
2 Irodalomkutatás, technológiák, hasonló alkotások bemutatása.....	11
2.1 Technikai háttér	11
2.2 Szerver oldali technológiák.....	11
2.2.1 ASP.NET Core.....	11
2.2.2 Hitelesítés / autorizáció.....	12
2.2.3 E-mail szolgáltatás	12
2.2.4 API dokumentáció és kliensgenerálás (Swagger és NSwag)	13
2.2.5 Entity Framework	13
2.3 Kliens oldali technológiák	14
2.3.1 Angular	14
2.3.2 TypeScript.....	14
2.3.3 HTML	15
2.3.4 Bootstrap.....	15
2.4 Adattárolás	16
2.4.1 Microsoft SQL Server.....	16
2.5 Fejlesztőkörnyezet	16
2.5.1 Visual Studio Code	16
2.5.2 Visual Studio 2022.....	16
2.5.3 Munkafolyamat követése	17
2.5.4 Verziókezelés (Git és GitHub).....	17
3 A feladatkiírás pontosítása és részletes elemzése	18
3.1 Funkcionális követelmények	18
3.2 Nem funkcionális követelmények.....	19
3.3 Szerepkörök és használati esetek	19
3.4 Rendszerarchitektúra	21

3.5	Adatbázis tervezés	22
3.6	Fejlesztési környezet és eszközök.....	22
4	Önálló munka bemutatása	23
4.1	Rendszertervezés és adatmodell	23
4.1.1	Adatbázis koncepció	23
4.1.2	Adatmodell részletezése:	24
4.1.3	DTO-k szerepe	26
4.2	Backend megvalósítása (ASP.NET Core)	26
4.2.1	Architektúra és tervezési minták.....	26
4.2.2	Hitelesítés és biztonság.....	27
4.2.3	Felhasználói profil és célok kezelése.....	29
4.2.4	Ételek	30
4.2.5	Aktivitás katalógus	31
4.2.6	Receptkezelés.....	31
4.2.7	Naplózás.....	32
4.2.8	Adatszolgáltatás a vizualizációhoz (Grafikon és Naptár).....	34
4.2.9	API Dokumentáció	35
4.3	A Frontend megvalósítása.....	35
4.3.1	Kliens oldali architektúra és modularitás.....	35
4.3.2	Navigáció és útvonal védelem (Routing & Security)	39
4.3.3	Kommunikáció a backenddel (API Layer)	42
4.3.4	Felhasználókezelés (Account Module).....	45
4.3.5	Adatkezelés és törzsadatok (CRUD)	47
4.3.6	Komplex űrlapkezelés: receptkészítő	49
4.3.7	Naplózás megvalósítása (Daily Note).....	51
4.3.8	Profil és statisztikák	57
4.3.9	Felhasználói élmény (UX) elemek	59
5	Önálló munka értékelése, mérések, tesztelés, eredmények bemutatása	62
5.1	Automatizált tesztelés és kódminőség	62
5.2	Eredmények és a követelmények teljesülése	63
6	Összefoglaló	64
7	Irodalomjegyzék.....	65

HALLGATÓI NYILATKOZAT

Alulírott **Pásztori Péter Benjám**n, szigorló hallgató kijelentem, hogy ezt a diplomatervet meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző, cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövegét pedig az egyetem belső hálózatán keresztül (vagy hitelesített felhasználók számára) közzétegye. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Kelt: Budapest, 2025. 12. 10.

.....
Pásztori Péter Benjám

Összefoglaló

A 21. század emberét a mozgásszegény életmód és a feldolgozott ételek komoly egészségügyi kihívások elé állították. Ennek egyik eredménye az, hogy felértékelődött a digitális egészségtudatosság. A felhasználók egyre inkább olyan szoftvereket igényelnek, amik a táplálkozást és aktivitást egyben kezelik. A jelenlegi piaci megoldások azonban nem nyújtanak kellő rugalmasságot az egyéni célok és a receptek közös használatában.

Diplomamunkám célja egy korszerű, egységesített webalkalmazás megvalósítása volt az egészséges életmód támogatására. A fejlesztés során „Full Stack” megközelítést alkalmaztam: a rendszert a nagyvállalati környezetben elterjedt ASP.NET Core és a skálázható Angular keretrendszerek ötvöztetésével építettem fel.

A szerveroldalon Microsoft SQL Server alapú, Code-First rendszert alakítottam ki, nagy figyelmet fordítottam az adatok teljességére. Implementáltam a „Recalculation Chain”(újra számolási lánc) üzleti logikát, amely valós időben, matematikailag helyesen frissíti a napi statisztikákat. A biztonságos működéshez az ASP.NET Core Identity, JWT hitelesítést és a Mailjet e-mail szolgáltatást alkalmaztam.

A fejlesztés során a kliens és a szerver közötti kommunikációt NSwag kódgenerátor bevezetésével növeltem, amellyel automatizáltam a frontend és backend közötti típusbiztos kommunikációt, így spórolva időt és energiát. A felhasználói élmény növeléséhez reaktív űrlapokat és Chart.js alapú vizualizációt használtam. A rendszer megbízhatóságát és az üzleti logika helyességét xUnit és InMemory Database alapú automatizált integrációs tesztekkel ellenőriztem.

Az elkészült alkalmazás sikeresen teljesíti a funkcionális követelményeket, stabil felületet biztosít a felhasználóknak. A jövőben a rendszert, vonalkód-leolvasóval és különböző bejelentkezési lehetőséggel tervezem tovább fejleszteni.

Abstract

In the 21st century, sedentary lifestyles and processed foods have presented people with serious health challenges. Consequently, digital health awareness has gained significant prominence. Users increasingly demand software solutions that manage nutrition and physical activity in an integrated manner. However, current market solutions lack sufficient flexibility regarding individual goals and the shared use of recipes.

The objective of my thesis was to implement a modern, unified web application to support a healthy lifestyle. During development, I applied a "Full Stack" approach: I built the system by combining the ASP.NET Core framework, widely used in enterprise environments, with the scalable Angular framework.

On the server side, I designed a Microsoft SQL Server-based Code-First system, paying close attention to data integrity. I implemented the "Recalculation Chain" business logic, which updates daily statistics in real-time with mathematical precision. To ensure secure operation, I utilized ASP.NET Core Identity, JWT authentication, and the Mailjet email service.

During development, I optimized client-server communication by introducing the NSwag code generator. This allowed me to automate type-safe communication between the frontend and backend, thereby saving significant time and effort. To enhance the user experience, I employed reactive forms and Chart.js-based data visualization. I verified the system's reliability and the correctness of the business logic using automated integration tests based on xUnit and an InMemory Database.

The completed application successfully meets the functional requirements and provides a stable interface for users. In the future, I plan to further develop the system by adding a barcode scanner and various login options.

1 Bevezetés

1.1 A digitális egészségtudatosság kora

A 21. században általánossá vált az ülőmunka, ami a fizikai aktivitás drasztikus csökkenésével járt. Ezzel egyidejűleg a mindennapi élet felgyorsult tempója és a folyamatos stressz, a könnyen elérhető, magasan feldolgozott élelmiszerek hozzájárultak a krónikus megbetegedések, mint például az elhízás, szív és érrendszeri problémák gyakori kialakulásához.

Ebben a környezetben az egészségtudatosság és a megelőzés szerepe felértékelődött. Az emberek egyre inkább felismerik azt, hogy az egészségük megőrzéséért aktívan kell tenniük, és nem csupán a már kialakult betegségeket kell kezelniük.

Ez a szemlélet hívta életre azt a technológiai robbanást, ami az öngondoskodást és az egészségügyi megelőzést digitális alapokra helyezte. A szélessávú internet elterjedése, az okostelefonok és a viselhető okoseszközök (mint az okosórák és fitnesskarkötők) mindennapivá válása gyökeresen megváltoztatta a viszonyunkat a saját testünk működéséhez.

Megjelent az úgynevezett "Quantified Self" (Számszerűsített én) mozgalom, aminek a központi gondolata az, hogy az egyének képesek adatokat gyűjteni saját életmódjukról (lépésszám, alvási ciklus, pulzusszám, tápanyagbevitel). Az okoseszközök lehetővé tették, hogy a biológiai adatok bárki számára azonnal elérhetővé és értelmezhetővé váljanak. A technológia mindenkinek elérhetővé tette az adathozzáférést, és ezzel megnőtt az igény az olyan alkalmazások iránt, amelyek ezt a nyers adathalmazt képesek értelmezni és egyúttal az egyén számára egy cselekvési tervet (mozgás, étkezés területén) kialakítani. [1]

A szoftveres megoldások már nem kizárólag passzív adatgyűjtők, hanem aktívan részt vesznek az életmódváltás folyamatában. Lehetővé teszik a felhasználók számára, hogy személyre szabott célokat tűzzenek ki. Lehet ez a napi kalóriabevitel, makro tápanyag egyensúly, a testsúly kívánt alakulása vagy a heti edzésterhelés.

Ezek a felületek vizuális visszajelzésekkel, statisztikákkal és napi összesítőkkal támogatják a felhasználói motivációt, és látványossá teszik a haladást. Kiemelt fontosságú az egészséges életmód két legalapvetőbb pillére: a táplálkozás és a fizikai aktivitás nyomon követése. Míg a viselhető eszközök az aktivitást egyre nagyobb pontossággal, automatikusan mérik, addig a táplálkozás naplózása – annak összetettsége miatt – továbbra is jelentős felhasználói beavatkozást és precíz szoftveres támogatást igényel.

1.2 A probléma felvetése és a fejlesztésem célja

Az informatikai megoldások jelentős segítséget nyújthatnak az egészségtudatosabb élet kialakításában. A digitális naplózás, a valós idejű visszajelzések és a statisztikai elemzések segíthetik a felhasználókat a tudatosabb döntéshozatalban. A diplomatervemben modern technológiák (ASP.NET Core és Angular) felhasználásával egy olyan webalkalmazás fejlesztését tűztem ki célul, amely alkalmas az egészséges életmód támogatására.

A dolgozatomban az alkalmazás fejlesztési folyamatát, szerkezetét mutatja be. Nagy figyelmet fordítottam a rendszer felépítésére, a technológiai döntésekre, a felhasználói funkciók megvalósítására. A rendszerem képes a felhasználó napi étkezéseinek, aktivitásainak, testsúlyának és célkitűzéseinek kezelésére. A fejlesztésem segítségével az összegzés és a vizualizálás is élményközpontúbb.

1.3 A megoldás szakmai motivációja

Az egészségtudatos életmód tőlem sem áll távol. Saját és baráti társaságom tapasztalati ösztönöztek a fejlesztésem kidolgozásában. Tapasztalataink szerint az aktivitási és étkezési adatok párhuzamos eszközökben, táblázatokban és különálló mobilalkalmazásokban történő vezetése gyakran pontatlan. A napi adatok kézi összesítése, a receptek és az egyedi ételek vegyes kezelése időigényes és nehézkes. Ezek oda vezethetnek, hogy a naplózások elmaradnak és a kezdeti lendület elveszik.

Jó ötletnek tűnt egy olyan, bárki számára egyszerűen használható, mégis bővíthető webes megoldás, ami egy helyen összefoglalja, kezeli a különböző funkciókat (étkezések, receptek, aktivitások). A fejlesztésem szemléletesen mutatja be az adatokat, a változásokat, így hozzájárul ahhoz, hogy követhetőbb és látványosabb legyen a haladás

(testtömeg növekedés, fogyás). A témaválasztásom szakmailag is indokolt, mert egy modern webes rendszerre épít, gyakorlati példákkal mutatja be, hogy illeszthető össze a felhasználói kényelem a típusú API kommunikáció és az adatvédelem követelménye.

1.4. A dolgozat felépítése

A bevezetést követően a 2. fejezetben a fejlesztéshez használt technológiákat és eszközöket mutatom be. Részletezem a szerveroldali (ASP.NET Core, Entity Framework Core) és a kliensoldali (Angular, TypeScript) keretrendszereket és indokolom a választásuk okát. Indokolom az adattároláshoz választott Microsoft SQL Server és a verziókövetéshez használt Git rendszer szerepét a projektben.

A 3. fejezetben dokumentálom a tervezési folyamatot. Meghatározom a rendszerrel szemben támasztott funkcionális és nem funkcionális követelményeket, elvárásokat, felhasználói szerepköröket. Mindezek után bemutatom az általam elképzelt és megvalósított rendszer szerkezetét és az adatbázis tervét.

A dolgozatom központi részét a 4. fejezet képezi. Itt az önálló fejlesztői munkámat mutatom be. Itt részletezem a végleges adatmodellt, a backend üzleti logikájának és a frontend felhasználói felületének a megvalósítását. Ismertetem a legfontosabb algoritmusokat (mint például a tápanyag számítása és a receptek kezelése) és kitérek a kliens és a szerver közötti biztonságos kommunikáció kiépítésére.

Az 5. fejezet az elkészült alkalmazást tartalmazza. Bemutatom a tesztelési folyamatot, eredményét. Indokolom, hogy az általam fejlesztett rendszer miért felel meg a feladatkiírásban és a tervezés során megfogalmazott követelményeknek.

A 6. fejezetben összegzem a munkám eredményét, a szakmai tapasztalataimat. Ebben a fejezetben fogalmazom meg az alkalmazás tovább fejlesztésének lehetséges irányát.

2 Irodalomkutatás, technológiák, hasonló alkotások bemutatása

2.1 Technikai háttér

A webalkalmazásomat két fő részre bontottam: a backendre és a frontendre. A szerveroldali logikát (backend) ASP.NET Core technológiával készítettem el, mivel ezt egy modern, moduláris és nyílt forráskódú keretrendszernek tartom. A kliensoldali megjelenítéshez (frontend) az Angular 19 keretrendszert választottam, ami erős JavaScript alapjaival különösen előnyösnek bizonyult az egyoldalas (SPA) alkalmazásom fejlesztéséhez.

A backend oldalon Microsoft SQL Server adatbázist alkalmaztam, amellyel az Entity Framework Core ORM révén kommunikálok. A rendszerben RESTful API-kon keresztül biztosítottam a kapcsolatot a frontend felé, így szigorúan szétválasztottam a kliens és a szerveroldali logikát. Data Transfer Object (DTO) használatával garantáltam, hogy a backend és a frontend közötti adattovábbítás tömör és biztonságos legyen.

A frontendnél az NSwag eszköz segítségével generált TypeScript service osztályokat használtam, melyek automatikusan illeszkednek az általam írt backend API-khoz és DTO-khoz. A felhasználói felületen Angular Material és Bootstrap komponenseket alkalmaztam az esztétikus megjelenés érdekében. A felhasználókezelést az ASP.NET Identity segítségével oldottam meg, amit kiegészítettem jelszó visszaállítási, email megerősítési és token alapú hitelesítési lehetőségekkel. Az email küldést a Mailjet API segítségével valósítottam meg.

2.2 Szerver oldali technológiák

2.2.1 ASP.NET Core

A diplomatervem szerveroldali logikáját, a RESTful API-t, ASP.NET Core keretrendszerrel valósítottam meg. Ez a modern, platformfüggetlen keretrendszer tökéletes alapot adott egy robusztus, nagy teljesítményű webes API elkészítéséhez.

A fejlesztés során kihasználtam a keretrendszer számos beépített funkcióját, amelyek közül a projektem szempontjából a legfontosabbak a következők voltak: A Kestrel amely egy beépített, nagy teljesítményű http kiszolgáló. A Függőséginjektálást (Dependency Injection) alkalmaztam a szolgáltatások (pl. RecipeService, DailyNoteService) esetében, ami a kódot tisztán és tesztelhetően tartotta, továbbá a beépített biztonsági funkciókra építettem a hitelesítési és autorizációs logikát. [2]

2.2.2 Hitelesítés / autorizáció

A felhasználók biztonságos kezelését és az API végpontok védelmét ASP.NET Identity segítségével oldottam meg.

A kliensoldali Angular alkalmazás és a backend közötti kommunikációhoz JSON Web Token (JWT) alapú hitelesítést alkalmaztam. A Microsoft.AspNetCore.Authentication.JwtBearer csomagot használtam a kliens által küldött tokenek validálására.

A felhasználók, szerepkörök (Roles) és a jelszavak (hash-elt formában) tárolását a Microsoft.AspNetCore.Identity.EntityFrameworkCore csomag tette lehetővé, amely automatikusan összekapcsolta az Identity modellt az Entity Framework adatbázis sémámba. A tokenek tényleges generálását a JWTService-ben, a IdentityModel.Tokens.JWT csomag segítségével oldottam meg. [3] [4]

2.2.3 E-mail szolgáltatás

A felhasználói regisztráció megerősítéséhez és az "elfelejtett jelszó" funkcióhoz szükség volt egy megbízható e-mail küldő szolgáltatásra. Erre a célra a Mailjet platformját választottam.

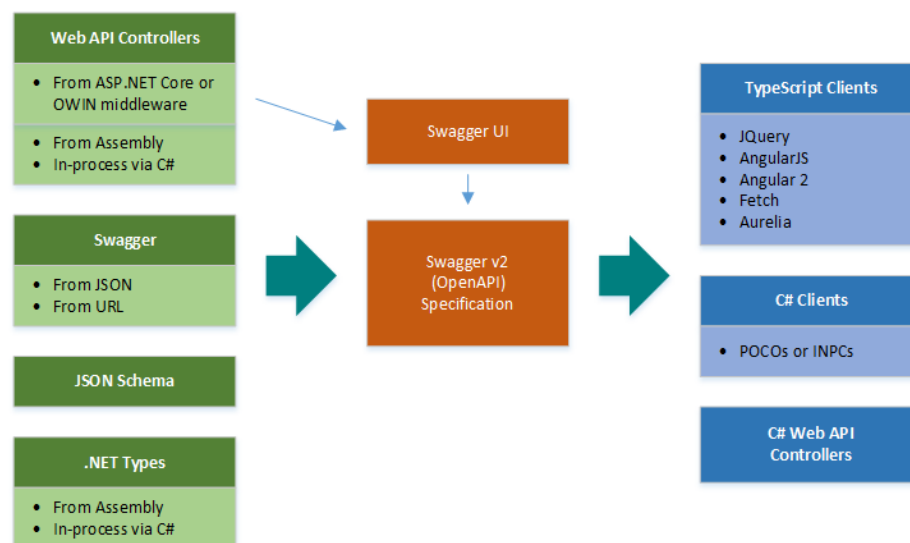
A HealthyAPI/Services/EmailService.cs osztályomban implementáltam azt a logikát, amely a Mailjet Representational State Transfer (REST) alapú API-ján keresztül küldi ki a leveleket. Az API hívások törzse JavaScript Object Notation (JSON) formátumú adatokat tartalmaz. [5]

2.2.4 API dokumentáció és kliensgenerálás (Swagger és NSwag)

A frontend és a backend fejlesztésének szinkronizálása és a típusbiztos kommunikáció érdekében kulcsfontosságú volt az API pontos definiálása. Erre az NSwag eszköztárat használtam.

Az NSwag a C# kontroller osztályaim (HealthyAPI/Controllers) és DTO-im (HealthyAPI/DTOs) alapján futásidőben automatikusan generálta az OpenAPI (Swagger) specifikációt. Ez a swagger.json fájl szolgált alapul a böngészőben futó interaktív Swagger UI felülethez, ami nagy segítség volt a fejlesztés és a tesztelés során.

A munkafolyamat másik kulcslépéseként az NSwag képes volt ebből az OpenAPI specifikációból legenerálni a teljes TypeScript kliens osztályt (Client/src/app/shared/models/Nswag generated/NswagGenerated.ts) az Angular alkalmazásom számára. Így a frontend oldalon típusbiztosan, fordítási időben ellenőrizhető metódusokkal hívhattam meg a backend végpontokat, elkerülve a manuális http hívásokból adódó hibákat. [6]



1. ábra Nswag generálás folyamata

2.2.5 Entity Framework

Az alkalmazás adatainak tárolásához Microsoft SQL Server adatbázist választottam. Az adatbázis és a C# objektumorientált modellek (HealthyAPI/Models) közötti leképezését az Entity Framework Core (EF Core) keretrendszerrel oldottam meg.

Ez a modern, objektum relációs leképező (ORM) lehetővé tette, hogy tiszta C# osztályokkal definiáljam az adatbázis sémáját, és ne kelljen manuálisan SQL parancsokat írnom. A lekérdezéseket Language Integrated Query (LINQ) segítségével adtam meg a szolgáltatási rétegben (pl. a RecipeService-ben). Az adatbázis séma változásait és verziókövetését az EF Core Migrations funkciójával kezeltem. [4]

2.3 Kliens oldali technológiák

A felhasználói felületet (frontend) modern, egyoldalas alkalmazásként (SPA - Single Page Application) terveztem meg. Ennek lényege, hogy a böngésző inicializáláskor csak egyszer tölti be a keretrendszert és az alkalmazás vázát, a továbbiakban pedig a felhasználói interakciók során dinamikusán cserélem a tartalmat, az oldal teljes újra töltése nélkül. A fejlesztés során komponens alapú architektúrát követtem, ami lehetővé tette a kód újra felhasználhatóságát és a könnyebb karbantarthatóságot.

2.3.1 Angular

A kliens oldali logikát az Angular 19 verziójára építve valósítottam meg. A projektet modulokra és komponensekre bontottam a Client/src/app könyvtárban.

A Routing modullal (AppRoutingModule) oldottam meg az oldalak közötti navigációt. A definiáltam az útvonalakat, amelyekhez a megfelelő komponenseket rendeltem hozzá, így a felhasználó számára az oldalváltás azonnali és zökkenőmentes. A Függőséginjektálás (Dependency Injection) tervezési mintát használtam a szolgáltatások (Services) beemelésére. A Reaktív űrlapokat (Reactive Forms) alkalmaztam a szigorúbb validációt igénylő felületekhez. Ez lehetővé tette a beviteli mezők komplex validációját (pl. kötelező mezők, e-mail formátum, jelszóerősség) még azelőtt, hogy az adatokat elküldtem volna a szervernek. [7]

2.3.2 TypeScript

A TypeScript egy magas szintű programozási nyelv, amely statikus típuskezelést és opcionális típus annotációkat ad a JavaScripthez. Bár a böngészők JavaScriptet futtatnak, a forráskódot TypeScript nyelven írtam. Ez lehetővé tette számomra a szigorú típusosság érvényesítését az egész alkalmazásban. Különösen nagy hasznát vettem az

NSwag eszközzel generált DTO-k esetében. Így, ha a backend megváltozott, a fordító azonnal jelezte a hibát a kliens kódban. [8] [9]

2.3.3 HTML

A HTML - azaz a Hypertext Markup Language - egy hiperszöveges leírónyelv. Egyfajta programozási előírás, ami megadja azt, hogyan is kell a weboldalakat írni úgy, hogy azokat megértse és megfelelően meg is jelenítse a számítógép. A neve a következőkre utal: Hypertext szöveg, ami a hipertérben (más szóval az interneten) található, mely egy formázatlan szöveg. A szöveget Markup language segítségével lehet formázni, szabványszerű jel és kódkészletet tartalmaz, amit a böngésző képes értelmezni. A HTML-t 1990 óta használják weboldalak tervezésére. Olyan szöveges fájlokról van szó, amiket akár szövegszerkesztővel is tudunk módosítani. Az ebben megtalálható szövegek és szimbólumok írják le a böngészők számára, hogyan legyen 12 megjelenítve a weboldal. Elsődleges célja a dokumentumok megjelenítése volt, nem pedig a sokoldalú formázás, stílus. A későbbiekben - 1996-ban - jelent meg a HTML 3.0, a nyelv olyan újítása, ahol már script is beágyazásra kerülhet, ezáltal az eddig statikus kódot már dinamikussá lehetett alakítani. Ekkor jelent meg a stílus is, amivel egyszerre érkeztek a tartalomformázási lehetőségek. A 90-es évek végén a HTML 4.0-ban jelent meg a nemzetközi karakterkészlet használat. 2011-ben jelent meg a HTML 5, amivel a beágyazott videó- és audio tartalmak kezelésére koncentráltak. [10]

2.3.4 Bootstrap

A felhasználói felület alapjait a Bootstrap 5 keretrendszerrel alkalmaztam a projektbe, mivel ez szerintem a világ legnépszerűbb nyílt forráskódú eszközkészlete a reszponzív, mobil első weboldalak fejlesztéséhez. Annak érdekében, hogy az alkalmazás minden eszközön (mobiltelefonon és asztali gépen is) megfelelően jelenjen meg, a rácsrendszer (Grid system) osztályait (pl. container, row, col-md-6) alkalmaztam a HTML sablonokban az elrendezések kialakítására. Ez modern színvilágot és egységes tipográfiát¹ adott a felületnek. [11]

¹Tipográfia: A nyomtatott és digitális szövegek vizuális megformálásának művészete, amely a betűkkel, szövegekkel és képek elrendezésével foglalkozik.

2.4 Adattárolás

2.4.1 Microsoft SQL Server

A Microsoft SQL Server egy robusztus, vállalati szintű relációs adatbázis kezelő rendszer (RDBMS). Az alkalmazás strukturált adatainak (felhasználók, naplóbejegyzések, receptek) tárolására Microsoft SQL Server adatbázist (a fejlesztés során LocalDB) alkalmaztam. A választást az indokolta, hogy ez az adatbázis kezelő rendkívül hatékonyan integrálható a .NET környezetbe és az általam használt Entity Framework Core ORM-be. A kapcsolatot a HealthyAPI/appsettings.json fájlban konfiguráltam a DefaultConnection kapcsolattal. [12]

2.5 Fejlesztőkörnyezet

A fejlesztés során a hatékonyság növelése érdekében két különböző, az adott technológiához legjobban illeszkedő fejlesztőkörnyezetet (IDE) használtam párhuzamosan.

2.5.1 Visual Studio Code

A kliens oldali (Angular) fejlesztéshez a Visual Studio Code kódszerkesztőt választottam. Ez a szerkesztő támogatást nyújt a TypeScript nyelvhez és a webes technológiákhoz. Beépített terminál segítségével futtattam az Angular CLI parancsokat (pl. ng generate component, ng serve). [13]

2.5.2 Visual Studio 2022

A szerver oldalt Visual Studio 2022 integrált fejlesztőkörnyezetben készítettem el. A választást az indokolta, hogy ez az eszköz nyújtja a legteljesebb támogatást a C# nyelvhez és a .NET ökoszisztémához. A NuGet csomagkezelőn keresztül telepítettem és frissítettem a backend működéséhez szükséges könyvtárakat (pl. Microsoft.EntityFrameworkCore, NSwag.AspNetCore). A Hibakeresés (Debugging) segítségével kerestem a hibákat és lépésről lépésre tudtam követni a backend kód futását. [14]

2.5.3 Munkafolyamat követése

2.5.4 Verziókezelés (Git és GitHub)

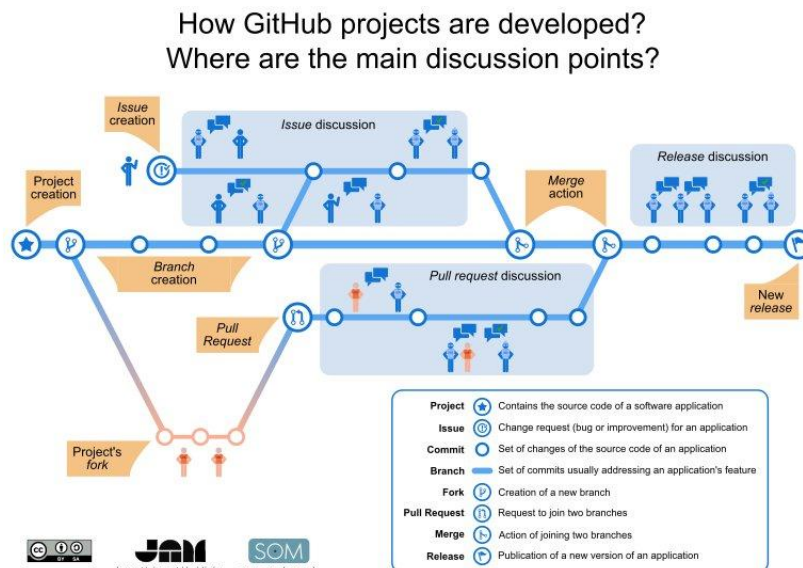
A forráskód biztonságos tárolását, verziókövetését és a változtatások dokumentálását a Git verziókezelő rendszerrel oldottam meg. A távoli tároló (remote repository) hosztolására a GitHub szolgáltatást vettem igénybe.

A fejlesztés során az alábbi munkafolyamatot követtem:

A Commit-ok segítségével a logikailag összetartozó módosításokat (pl. "Új recept létrehozása funkció hozzáadása") rendszeresen rögzítettem (commit), így a fejlesztés bármely pontjára visszatérhettem volna hiba esetén.

A Mappastruktúrát úgy védtem, hogy a gyökérkönyvtárban és a Client mappában elhelyezett .gitignore fájlok segítségével biztosítottam, hogy a felesleges, generált fájlok (pl. node_modules, bin, obj mappák, felhasználói beállítások) ne kerüljenek fel a felhőbe, csak a tiszta forráskód.

A Szinkronizáció segítségével a lokális gépen végzett munkámat rendszeresen szinkronizáltam (push) a GitHub tárolóval, ami nemcsak biztonsági mentésként szolgált, hanem lehetővé tette a kód hordozhatóságát is a különböző fejlesztői gépek között. [15]



2. ábra GitHub munkafolyamat

3 A feladatkiírás pontosítása és részletes elemzése

3.1 Funkcionális követelmények

A funkcionális követelmények határozzák meg a rendszerem elvárt viselkedését és szolgáltatásait. A feladatkiírás és a saját terveim alapján az alábbi funkciókat határoztam meg:

- Felhasználókezelés:
 - Regisztráció név, e-mail cím és jelszó megadásával.
 - Biztonságos bejelentkezés (JWT token alapú hitelesítés).
 - E-mail cím megerősítése és elfelejtett jelszó visszaállítása (külső e-mail szolgáltatóval).
 - Felhasználói profil kezelése (név, életkor, nem, magasság, súly, testzsír százalék, heti aktivitási szint kiválasztása és a célsúly megadása).
- Aktivitás követés:
 - Új aktivitás (Activitycatalog) felvitele (aktivitás neve, kalória és a hozzá tartozó időegység).
 - Aktivitás listázása, szerkesztése és törlése.
- Ételek és Receptek kezelése:
 - Új ételek (Food) felvitele tápérték adatokkal (étel neve és tömege grammban, kalória, makrók,).
 - Komplex receptek (Recipe) összeállítása több alapanyagból, ahol a rendszer automatikusan kiszámolja az összesített tápértéket(neve, leírás, milyen étel és milyen mennyiségben).
 - Saját ételek és receptek listázása, szerkesztése és törlése.
- Naplózás (Daily Note):
 - Gyorskeresés az ételek és receptek között név alapján

- Napi étkezések rögzítése étkezéstípusonként (pl. reggeli, ebéd).
- A kiválasztott ételek/receptek hozzáadása a naplóhoz a fogyasztott mennyiség megadásával.
- Aktuális napi testsúly rögzítése.
- A napi bevitel (kalória, makrók) valós idejű összesítése és vizuális összevetése a beállított céllal.
- Mozgásformák kiválasztása katalógusból és a hozzá tartozó időegység megadása
- Statisztika és Vizualizáció:
 - Naptár funkció a korábbi napok adatainak visszakeresésére.
 - A testsúly alakulásának grafikonos megjelenítése az idő függvényében.

3.2 Nem funkcionális követelmények

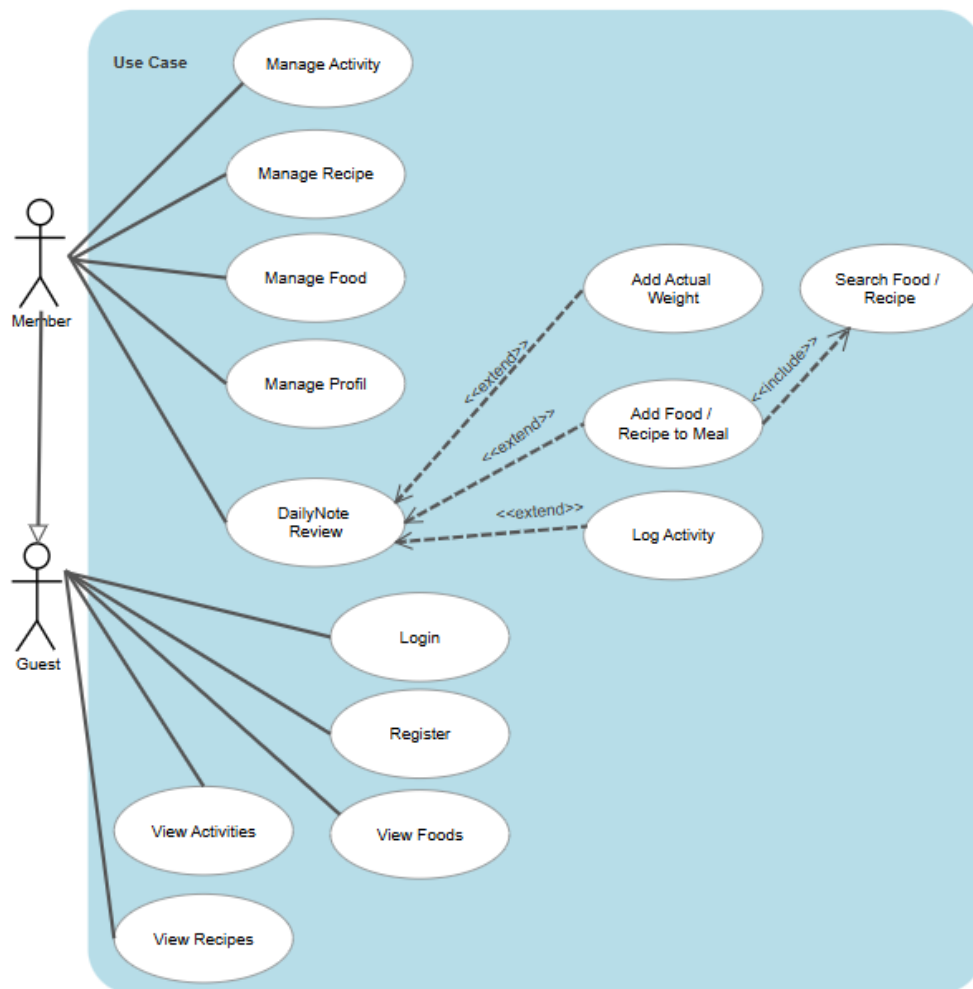
A rendszerrel szemben támasztott minőségi követelményeket az alábbiakban határoztam meg:

- Biztonság: A jelszavakat nem olvasható formában, hanem hash-elve tárolom az adatbázisban (ASP.NET Identity), a végpontokat pedig hitelesítés védi.
- Teljesítmény: Az alkalmazásnak gyors válaszidőt kell biztosítania. Ennek érdekében aszinkron adatbázis műveleteket és kliensoldali renderelést (SPA) alkalmaztam.
- Használhatóság: A felületnek reszponzívnak kell lennie, hogy asztali számítógépen és más eszközön is kényelmesen használható legyen. Ezt a Bootstrap keretrendszerrel biztosítottam.

3.3 Szerepkörök és használati esetek

A rendszerben a jelenlegi megvalósításban két fő szerepkört különböztettem meg:

- Vendég (Guest): Nem regisztrált látogató jogosultsága korlátozott. A kezdőoldalt (Food), az Aktivitásoldalt (Activitycatalog), a Recepteket (Recipes), továbbá a regisztrációt és a bejelentkezést éri el.
- Regisztrált felhasználó (Member): Bejelentkezett felhasználó, aki hozzáfér a rendszer teljes funkcionalitásához: vezetheti a naplóját, létrehozhat saját ételeket és recepteket, továbbá sport tevékenységeket, valamint követheti a súlyának alakulását és menedzselheti a profilját a céljának megfelelően.



3. ábra Use Case diagram

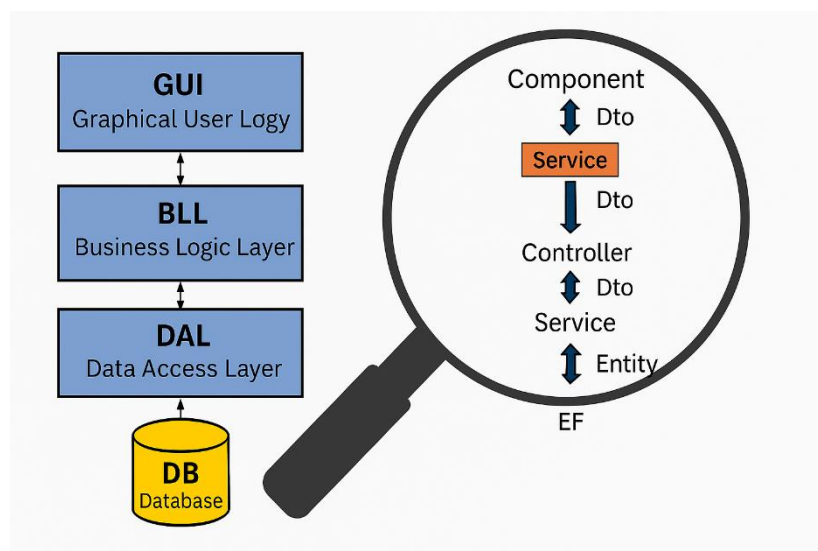
A legfontosabb használati eset, a "Napi naplózás" folyamata a következőképpen néz ki: a felhasználó kiválasztja az adott napot, majd az étkezés típusát (pl. Ebéd). A kereső segítségével kiválaszt egy ételt vagy receptet, megadja a mennyiséget, és menti a bejegyzést. A rendszer ezt követően azonnal frissíti a napi összesítő sávokat. Ezen túlmenően a felhasználó megadhatja a napi aktivitását és aznapi testsúlyát is.

3.4 Rendszerarchitektúra

A rendszert a modern webes fejlesztésben elterjedt kliens-szerver architektúrára építettem. A megoldás két fizikailag és logikailag is elkülönülő komponensből áll:

- Frontend (Kliens): Egy Angular alapú egyoldalas alkalmazás (SPA), amely a felhasználó böngészőjében fut. Feladata a felhasználói interakciók kezelése és az adatok megjelenítése.
- Backend (Szerver): Egy ASP.NET Core Web API, amely az üzleti logikát valósítja meg, és kapcsolatot tart az adatbázissal.
- Külső szolgáltatások (3rd Party Services): A regisztrációs folyamat és a biztonságos jelszókezelés támogatására a rendszer egy külső e-mail küldő szolgáltatást (Mailjet) integrál. Ez a komponens nem része a belső hálózatnak, a backend REST API hívásokon keresztül kommunikál vele a tranzakcionális e-mailek kiküldéséhez.

A két réteg között a kommunikáció REST (Representational State Transfer) alapú HTTP kérésekkel történik, JSON adatformátumban. Ez a laza csatolás biztosítja, hogy a kliens és a szerver egymástól függetlenül fejleszthető legyen.



4. ábra Rendszerarchitektúra diagram

3.5 Adatbázis tervezés

Az adatok tárolására relációs adatbázist terveztem (Microsoft SQL Server). A tervezés során a Code-First megközelítést alkalmaztam, azaz először a C# modellosztályokat (Models mappa) hoztam létre, és ebből generáltam le az adatbázis sémát az Entity Framework Core segítségével.

A rendszer főbb entitásai és kapcsolatai:

- User: A központi entitás, amelyhez minden személyes adat (naplók, receptek) kapcsolódik.
- DailyNote (Napi napló): A naplózás fő pillére. A kapcsolatokat úgy alakítottam ki, hogy egy DailyNote-hoz tetszőleges számú étkezés (MealEntries) és sporttevékenység (UserActivity) tartozhasson (1:N kapcsolat). Ez teszi lehetővé, hogy a felhasználó egy napon belül több különböző mozgásformát (pl. futás, konditerem) is rögzíthessen.
- MealEntries (Étkezés): Az étkezéseket úgy terveztem, hogy azok ne csak egyetlen elemet tartalmazhassanak. A MealFoods és MealRecipes kapcsolótáblák segítségével egyetlen étkezéshez (pl. Ebéd) több különböző étel és recept is hozzárendelhető.
- Recipe és Food: A receptek és az ételek közötti kapcsolatot egy dedikált kapcsolótáblával (RecipeFoods) valósítottam meg. Mivel egy recept számos alapanyagból állhat, és egy alapanyag (pl. tojás) számtalan receptben szerepelhet, itt több a többhöz (N:M) kapcsolatot használtam.

3.6 Fejlesztési környezet és eszközök

A fejlesztés során a verziókezelést Git rendszerrel oldottam meg, a forráskódot pedig a GitHub felhőszolgáltatásban tároltam. Ez lehetővé tette a változások nyomon követését és a biztonsági mentést. A fejlesztői környezetként a Visual Studio (backend) és a Visual Studio Code (frontend) fejlesztői környezeteket használtam, amelyek hatékony hibakeresési és kódkiegészítési funkciókkal támogatták a munkámat.

4 Önálló munka bemutatása

A diplomaterv legfontosabb részeként ebben a fejezetben ismertetem az általam fejlesztett webalkalmazás konkrét megvalósítását. Részletezem a tervezési döntéseimet, az adatbázis felépítését, valamint a szerver és kliensoldali üzleti logika megvalósítását.

4.1 Rendszertervezés és adatmodell

A fejlesztés első lépéseként az adatok szerkezetét és a rendszer alapvető működési logikáját határoztam meg. Mivel az alkalmazás központi eleme a táplálkozási adatok és a felhasználói naplók pontos kezelése, elengedhetetlen volt egy robusztus és jól strukturált adatmodell kialakítása.

4.1.1 Adatbázis koncepció

Az adatok tárolására Microsoft SQL Server adatbázis kezelőt választottam. A döntésemet az indokolta, hogy technológia ez integrálható legjobban a .NET ökoszisztémába, és jó eredményeket mutat a relációs adatkezelésben.

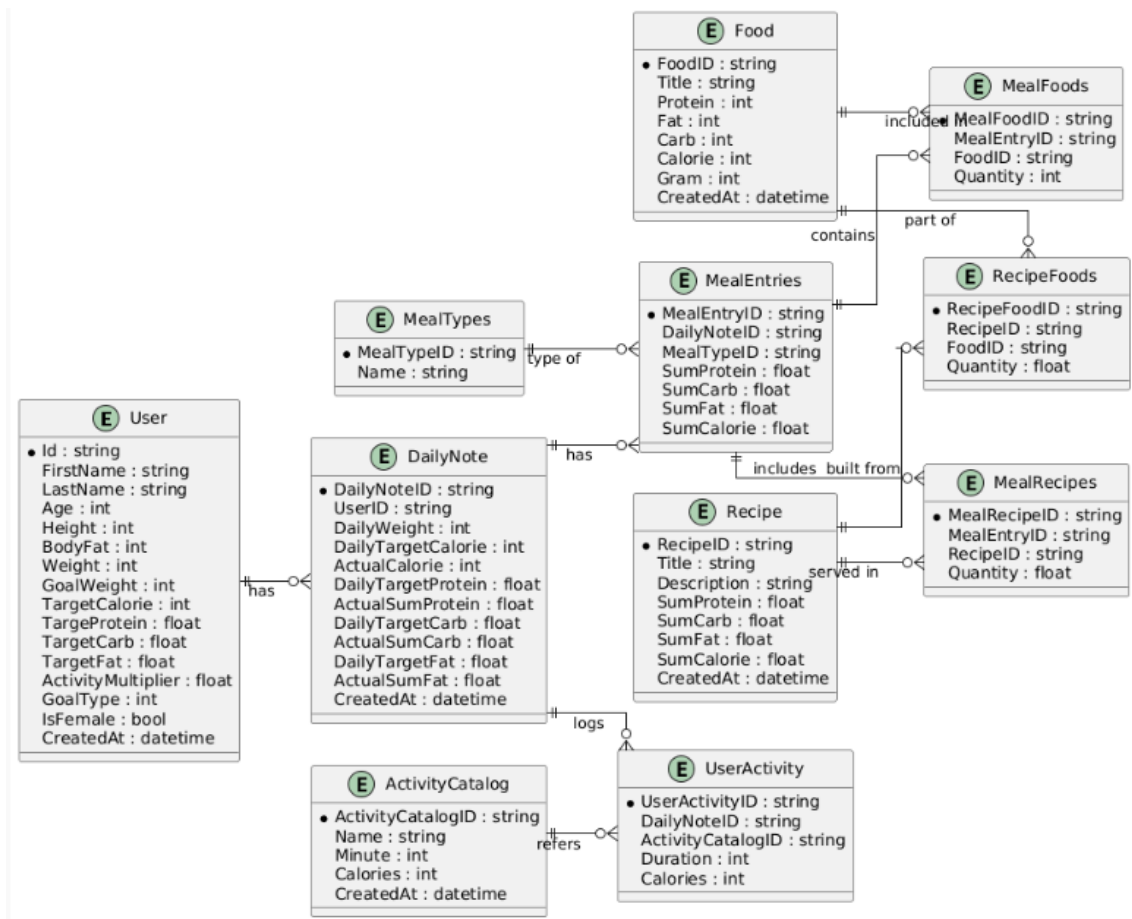
A fejlesztés során a Code-First megközelítést alkalmaztam az Entity Framework Core (EF Core) keretrendszer segítségével. Ez a módszer lehetővé tette, hogy az adatbázis sémáját közvetlenül C# osztályokként (modellekként) definiáljam a kódban (HealthyAPI/Models), majd ezekből generáljam le az adatbázist.

Ez a megközelítés számos előnnyel járt a munkám során:

- Verziókövetés: Az adatbázis séma változásait (Migrations) a kóddal együtt tudtam verzió kezelni a Git rendszerben.
- Típusbiztonság: A C# nyelv szigorú típusellenőrzése már a kódolás során segített elkerülni az adatkezelési hibákat, garantálta az adatok egységességét.
- Gyorsaság: Nem kellett SQL kérést kézzel írnom, az EF Core automatikusan elvégezte a táblák és kapcsolatok létrehozását a DbContext konfigurációim alapján.

4.1.2 Adatmodell részletezése:

Az alkalmazás adatmodelljét úgy terveztem meg, hogy a felhasználók könnyen össze tudják kapcsolni az ételeket, a recepteket és a napi tevékenységeket.



5. ábra A megvalósított rendszer Entitás kapcsolat (ER) diagramja

A entitások és szerepük a rendszerben:

User (Felhasználó): Ez a központi entitás, amely kiterjeszti az ASP.NET Identity alapértelmezett felhasználói modelljét. A HealthyAPI/Models/User.cs fájlban olyan egyedi tulajdonságokat határoztam meg, mint az Age (kor), Height (magasság), Weight (jelenlegi súly), BodyFat (Testzsír), IsFemale (milyen nemű), ActivityMultiplier (heti aktivitás szint) és a GoalWeight (célsúly). Itt tárolom a felhasználó napi makrotápanyag céljait is (TargetCalorie, TargetProtein stb.), amelyeket a rendszer a regisztrációkor vagy a profil szerkesztésekor számol ki.

DailyNote (Napi napló): A naplózás gerince. Egy felhasználónak minden naphoz egyetlen DailyNote bejegyzése tartozhat. Ez az entitás tárolja az adott napra vonatkozó összesített értékeket (ActualCalorie, DailyWeight stb.) és a kapcsolatokat az étkezésekhez továbbá az aktivitáshoz.

MealEntries (Étkezés): A DailyNote alatt helyezkedik el, és az egyes étkezéseket reprezentálja (pl. Reggeli, Ebéd). Egy naplóhoz több étkezés is tartozhat (1:N kapcsolat).

Food (Étel) és Recipe (Recept): Az ételeket a Food entitás, az ezekből összeállított fogásokat a Recipe entitás reprezentálja. Mindkettő tartalmazza a tápérték adatokat (makrókat továbbá a mennyiségüket és a recept esetében a recept leírást is).

RecipeFoods (Recept összetevők): Mivel egy recept több alapanyagból állhat, és egy alapanyag több receptben is szerepelhet, a Recipe és Food között N:M (sok a sokhoz) kapcsolat áll fenn. Ezt a RecipeFoods kapcsolótáblával oldottam meg, amely a mennyiséget (Quantity) is tárolja.

MealFoods és MealRecipes (Étkezés tételek): Hasonlóan a receptekhez, az egyes étkezések (MealEntries) tartalmát is kapcsolótáblákkal valósítottam meg. A MealFoods az egyszerű alapanyagokat, míg a MealRecipes a kész recepteket rendeli hozzá az adott étkezéshez, tárolva az elfogyasztott mennyiséget is. A kapcsolatot egy a többhöz (1:N) viszonyként alakítottam ki, ami lehetővé teszi, hogy egyetlen étkezéshez (pl. Ebéd) tetszőleges számú ételt és receptet is rögzíthessen a felhasználó vegyesen (pl. egy kész recept mellé külön felvihet még egy gyümölcsöt is), a rendszer pedig ezeket egyetlen étkezésként kezeli és összegzi.

MealTypes (Étkezéstípusok): Ez az entitás definiálja a választható étkezési kategóriákat (pl. Reggeli, Ebéd, Vacsora). A rendszert úgy terveztem, hogy egy naphoz (DailyNote) több étkezés (MealEntries) is tartozhasson, amelyek mindegyike hivatkozik egy egy étkezéstípusra. Ez teszi lehetővé, hogy a felhasználó elkülönítve, kategóriánként rögzíthesse és követhesse nyomon a napi tápanyagbevitelét.

UserActivity és ActivityCatalog (Aktivitások): A táplálkozás mellett a rendszer a sporttevékenységeket is kezeli. Az ActivityCatalog tartalmazza a választható mozgásformákat (és azok kalóriaégetési adatait). A UserActivity entitás kapcsolja össze ezeket a felhasználó napi naplójával (DailyNote). A kapcsolatot egy a többhöz (1:N) viszonyként modelleztem, ami biztosítja, hogy egy adott naphoz (DailyNote) tetszőleges

számú, különböző tevékenységet is rögzíthessen a felhasználó (pl. külön egy futást és egy úszást), a rendszer pedig ezeket külön külön tárolja, de az elégetett kalóriákat napiszinten összesíti.

4.1.3 DTO-k szerepe

A fejlesztés során szigorúan elválasztottam az adatbázis modelleket (amiket fentebb ismertettem) a külvilág felé kommunikált adatoktól. Erre a célra Data Transfer Object-eket (DTO) hoztam létre a HealthyAPI/DTOs mappában.

Ennek a rétegnek a bevezetése több okból is kritikus volt:

- Adatvédelem: Nem akartam minden adatot kiküldeni a kliensnek (pl. a felhasználó jelszavának hash-ét vagy belső azonosítókat). A UserDto például csak a megjelenítéshez szükséges adatokat (név, token) tartalmazza.
- Kliens specifikus formázás: A DTO-kban lehetőségem volt az adatokat a frontend igényei szerint formázni, anélkül, hogy az adatbázis szerkezetét módosítanom kellett volna.

A modellek és DTO-k közötti megfeleltetést (mapping) manuálisan végeztem a Service rétegben, hogy teljes ellenőrzésem legyen az adatok irányítása fölött.

4.2 Backend megvalósítása (ASP.NET Core)

Az alkalmazás szerveroldali logikáját ASP.NET Core Web API segítségével oldottam meg. A fejlesztés során a legfontosabb szempont a tiszta, karbantartható kódszerkezet kialakítása és a biztonságos adatkezelés volt. Ebben az alfejezetben részletezem az architektúrális döntéseimet, a hitelesítési folyamatot (üzleti logikát a profilkezeléstől kezdve a receptek összeállításán át a napi naplózásig).

4.2.1 Architektúra és tervezési minták

A backend fejlesztése során a szoftverfejlesztésben elterjedt, többretegű (N-tier) architektúrát követtem. Nagy figyelmet fordítottam a felelősségi körök szétválasztására. A Service (Szolgáltatás) réteget alkalmaztam az üzleti logika és az adatbázis elérés (Controller és DbContext) szétválasztására.

A rétegek közötti laza csatolást a függőséginjektálás tervezési minta alkalmazásával értem el. A Program.cs fájlban regisztráltam az összes általam írt szolgáltatást és azok interfészeit. Szolgáltatások regisztrációját scoped élettartammal állítottam be, ami biztosítja, hogy minden HTTP kéréshez új példány jöjjön létre, ezzel optimalizálva az erőforrás használatot.

```
builder.Services.AddScoped<IFoodService, FoodService>();
builder.Services.AddScoped<IMealFoodsService, MealFoodsService>();
builder.Services.AddScoped<IMealTypeService, MealTypeService>();
builder.Services.AddScoped<IRecipeFoodService, RecipeFoodService>();
builder.Services.AddScoped<IRecipeService, RecipeService>();
builder.Services.AddScoped<IMealRecipesService, MealRecipesService>();
builder.Services.AddScoped<IMealEntriesService, MealEntriesService>();
builder.Services.AddScoped<IActivityCatalogService,
ActivityCatalogService>();
builder.Services.AddScoped<IUserActivityService, UserActivityService>();
builder.Services.AddScoped<IUserProfileService, UserProfileService>();
builder.Services.AddScoped<IDailyNoteService, DailyNoteService>();
```

Ez a megoldás lehetővé tette, hogy a Controllerek (pl. DailyNoteController) csak az interfészekre támaszkodjanak, azaz ne függjenek a szolgáltatások konkrét implementációjától, csak az interfészektől. A Controllerek feladata kizárólag a HTTP kérések fogadása, a bemeneti adatok (DTO-k) validálása és a megfelelő szolgáltatás meghívása lett, míg a komplex számítási logika a Service rétegbe került.

4.2.2 Hitelesítés és biztonság

Mivel az alkalmazás érzékeny egészségügyi adatokat (súly, testzsír, étkezési szokások) kezel, a biztonságos hitelesítés kiemelt fontosságú volt. A rendszert ASP.NET Core Identity alapokon építettem fel, amelyet JWT (JSON Web Token) alapú tokenes hitelesítéssel egészítettem ki a API kommunikáció érdekében.

ASP.NET Core Identity konfiguráció: A Program.cs fájlban meghatároztam a felhasználókezelés szabályait. Az AddIdentityCore<User> szolgáltatás regisztrálásakor szigorú, de felhasználóbarát jelszó feltételt állítottam be (pl. minimum 6 karakter hossz), és biztonsági okokból bekapcsoltam a SignIn.RequireConfirmedEmail = true opciót, ami megakadályozza a bejelentkezést, amíg a felhasználó nem igazolja vissza az e-mail címét. Az adattároláshoz az AddEntityFrameworkStores<Context>() metódussal kötöttem össze az Identity rendszert az SQL adatbázissal.

4.2.2.1 A regisztráció és bejelentkezés folyamata

A felhasználókezelést az AccountController valósítja meg. A folyamat során Data Transfer Object-eket (DTO) használok az adatminimalizálás és a típusbiztonság érdekében.

Regisztráció: A kliens egy RegisterDto-t küld (Név, Email, Jelszó). A rendszer létrehozza a felhasználót, majd azonnal elindít egy e-mail megerősítési folyamatot.

Bejelentkezés: A LoginDto fogadása után a SignInManager ellenőrzi a hitelesítő adatokat. Sikeres azonosítás esetén a válasz egy UserDto, amely már tartalmazza a generált JWT tokenet is, így a kliens azonnal, további kérések nélkül hitelesített állapotba kerül.

4.2.2.2 E-mail szolgáltatás (Mailjet)

A regisztrációs és „elfelejtett jelszó” folyamatokhoz (ResetPasswordDto) külső e-mail küldő szolgáltatót, a Mailjet-et integráltam, amit az EmailService osztályba szerveztem ki és Scoped élettartammal regisztráltam a Program.cs-ben. A megvalósítás során a Mailjet.Client könyvtárat a levelek összeállítására. Ez lehetővé tette, hogy a HTML tartalmú e-maileket strukturáltan, C# kódból építsem fel, majd aszinkron módon (SendTransactionalEmailAsync) küldjem el a Mailjet API-nak JSON formátumban.

4.2.2.3 JWT Token generálás és Validáció

A sikeres bejelentkezés kulcsa a JWTService, amely felelős az aláírt tokenek létrehozásáért. A Program.cs-ben konfiguráltam a JwtBearer hitelesítést, ahol beállítottam a token érvényesítési paramétereit, hogy csak a saját szerverem által aláírt tokeneket fogadja el az API.

```

public string CreateJWT(User user)
{
    var userClaims = new List<Claim>
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id),
        new Claim(ClaimTypes.Email, user.UserName),
        new Claim(ClaimTypes.GivenName, user.FirstName),
        new Claim(ClaimTypes.Surname, user.LastName),
    };
    var credentials = new SigningCredentials(_jwtKey,
SecurityAlgorithms.HmacSha256Signature);
    var tokenDescriptor = new SecurityTokenDescriptor
    {
        Subject = new ClaimsIdentity(userClaims),
        Expires = DateTime.UtcNow.AddDays(int.Parse(_config["JWT:ExpiresInDays"])),
        SigningCredentials = credentials,
        Issuer = _config["JWT:Issuer"]
    };
    var tokenHandler = new JwtSecurityTokenHandler();
    var jwt = tokenHandler.CreateToken(tokenDescriptor);
    return tokenHandler.WriteToken(jwt);
}

```

A fenti kódban látható, hogy a tokenben szabványos Claim-ként (állításként) beégetem a felhasználó egyedi azonosítóját (NameId) és e-mail címét. Ez teszi lehetővé a szerver számára, hogy a későbbi kéréseknél adatbázis lekérdezés nélkül, pusztán a token dekódolásával azonosítsa a felhasználót (User.FindFirst(ClaimTypes.NameIdentifier)), jelentősen csökkentve ezzel a terhelést.

4.2.3 Felhasználói profil és célok kezelése

Az alkalmazás „agya” a felhasználói profil, amely alapján a rendszer a napi célokat kalkulálja. Ezt a komplex üzleti logikát a UserProfileService osztályban valósítottam meg.

A felhasználó a regisztrációkor vagy a profil szerkesztésekor megadja fizikai paramétereit és aktivitási szintjét. A rendszer ezekből az adatokból automatikusan számolja újra a szükségleteket.

A számítás lépései a következők:

Alapanyagcsere (BMR) meghatározása: A nemzetközileg elismert Mifflin-St Jeor képletet alkalmaztam, amely a legpontosabb becslést adja a nyugalmi energiafelhasználásra. A képlet figyelembe veszi a nemek közötti eltérést is (férfiaknál +5, nőknél -161 korrekciós tényezővel).

Napi energiaszükséglet (TDEE): A kapott BMR értéket megszoroztam a felhasználó által választott aktivitási szorzóval (ActivityMultiplier), amely pl. 1.2 (ülő életmód), 1.9 (extrém aktivitás) értékek lehetnek.

Célok szerinti korrekció: A felhasználó céljától (GoalType) függően módosítottam a kalóriakeretet: fogyás (Cutting) esetén 500 kalóriát vontam le, míg tömegnövelés (Bulking) esetén 500 kalóriát adtam hozzá a napi szükséglethez a fenntartható változás érdekében.

Makrotápanyagok felosztása: A makrók meghatározásánál a sporttáplálkozásban elterjedt, testsúly alapú megközelítést alkalmaztam. A fehérjebevitelt a testsúly kétszeresében (2g/kg), a zsírbevitelt a testsúly egyszeresében (1g/kg) rögzítettem. A fennmaradó kalóriakeretet a szénhidrátok fedezik, amelyet a rendszer automatikusan kalkulál a kalóriaértékek (fehérje: 4 kcal/g, zsír: 9 kcal/g, szénhidrát: 4 kcal/g) figyelembevételével.

A profil mentésekor a rendszerem ellenőrzi, hogy létezik-e már a mai napra vonatkozó naplóbejegyzés. Amennyiben igen, azonnal frissíti a mai célértékeket is, így a felhasználó rögtön az új beállítások szerint látja a hátralévő keretét.

4.2.4 Ételek

A rendszer alap egységei az ételek, amelyekből később a felhasználók étkezéseket és recepteket állíthatnak össze. Ezt az adatkezelési logikát a FoodController és FoodService osztályok együttműködésével valósítottam meg.

A felhasználó új étel felvételekor megadja az alapanyag nevét és pl 100 grammra vetített tápérték adatait (fehérje, zsír, szénhidrát, kalória). A rendszer ezekből a nyers adatokból egy többlépcsős folyamat során hozza létre a végleges adatbázis bejegyzést.

A DTO adataiból a rendszer példányosítja a végleges Food entitást. Ebben a lépésben történik meg az üzleti objektum technikai mezőinek például a létrehozás pontos idejét jelölő CreatedAt dátumbélyegnek az beállítása is.

Az előkészített entitást a FoodService osztály aszinkron módon adja hozzá a DbContext-hez, majd a SaveChangesAsync metódussal véglegesíti a tranzakciót az SQL adatbázisban. A sikeres mentést követően a rendszer egy FoodResponseDto objektumba

csomagolva adja vissza a létrehozott elem adatait (beleértve a generált azonosítót is), így a kliensoldali felület azonnal, képes megjeleníteni az új alapanyagot.

4.2.5 Aktivitás katalógus

A táplálkozás mellett az energia-gazdálkodáshoz tartozik sporttevékenységek is (Valójában maga az étel fogyasztása is energia fogyasztással jár de ez annyira elenyésző, hogy nem aktualizáltam a rendszeremben). A rendszerem egy katalógusban tárolja, amelyből a felhasználók később kiválaszthatják az elvégzett mozgásformákat. Ezt az adatkezelési logikát az ActivityCatalogController és ActivityCatalogService osztályokkal valósítottam meg.

Új tevékenységtípus felvételekor a megadható a mozgásforma neve, időtartam (percben) és az ezalatt elégetett kalóriamennyiség. A beérkező DTO (ActivityCatalogCreateDto) adataiból a szolgáltatás példányosítja a ActivityCatalog entitást. Ebben a lépésben történik meg az üzleti objektum technikai mezőinek – például a létrehozás pontos idejét jelölő CreatedAt dátumbélyegnek – az automatikus beállítása is.

A ActivityCatalogService osztály aszinkron módon adja hozzá a DbContext-hez, majd a SaveChangesAsync metódussal véglegesíti a tranzakciót az SQL adatbázisban. A sikeres mentést követően a rendszer egy ActivityCatalogResponseDto objektumba csomagolva adja vissza a létrehozott elem adatait (beleértve a generált azonosítót is), így a kliensoldali felület képes azonnal megjeleníteni az új mozgásformát a listában.

4.2.6 Receptkezelés

A rendszer egyik legösszetettebb algoritmus a receptek összeállítása és tápérték számítása. A követelmény az volt, hogy a felhasználó szabadon válogathasson össze alapanyagokat (Food), a rendszer pedig ezekből egyetlen, „Recept” entitást hozzon létre, automatikusan kalkulált tápértékekkel. Ezt a logikát a RecipesController és a RecipeService rétegekben valósítottam meg.

A folyamat vezérlése: A kérés a RecipesController Create végpontjára érkezik egy RecipeCreateDto formájában, amely tartalmazza a recept nevét, leírását, valamint a kiválasztott összetevők listáját és mennyiségeit. A vezérlő a kérést közvetlenül továbbítja az üzleti logikai rétegnek.

A komplex feldolgozást a RecipeService osztály Create metódusában valósítottam meg. A metódus az alábbi lépéseket hajtja végre:

Példányosítom az új Recipe objektumot, és beállítom az alapvető adatait (Cím, Leírás, Létrehozás dátuma). Egy ciklus segítségével végigiterálok a kapott összetevőkön (Ingredients). Minden egyes tételnél lekérdezem az adatbázisból a hozzá tartozó Food entitást, hogy hozzáférjek a tápértékekhez.

```
private async Task RecalculateNutrition(Recipe recipe)
{
    var recipeFoods = await _context.RecipeFoods
        .Include(rf => rf.Food)
        .Where(rf => rf.RecipeID == recipe.RecipeID).ToListAsync();
    recipe.SumProtein = (float)Math.Round((double)recipeFoods
        .Sum(rf => rf.Quantity / 100 * rf.Food.Protein),2);
    recipe.SumCarb = (float)Math.Round((double)recipeFoods
        .Sum(rf => rf.Quantity / 100 * rf.Food.Carb),2);
    recipe.SumFat = (float)Math.Round((double)recipeFoods
        .Sum(rf => rf.Quantity / 100 * rf.Food.Fat),2);
    recipe.SumCalorie = recipeFoods
        .Sum(rf => rf.Quantity / 100 * rf.Food.Calorie);
}
```

A ciklus belsejében végzem el a matematikai számításokat: az alapanyagok 100 grammra vetített értékeit a felhasználó által megadott mennyiséggel (Quantity) súlyozom, majd ezeket folyamatosan hozzáadom a recept összesített makróihoz (fehérje, zsír, szénhidrát, kalória), amelyet 2 tizedes jegyre kerekítetek.

A számítás mellett minden lépésben létrehozok egy RecipeFoods kapcsoló rekordot is, amely rögzíti, hogy az adott recepthoz melyik ételből mennyi tartozik.

Végezetül a kész Recipe entitást (a hozzárendelt RecipeFoods listával együtt) egyetlen tranzakcióban mentem el az adatbázisba a SaveChangesAsync hívással. Ezzel a megoldás rendszer automatikusan végzi el a makrotápanyagok összesítését a kiválasztott alapanyagok alapján.

4.2.7 Naplózás

Az alkalmazás központi funkciója a napi táplálkozás és aktivitás nyomon követése. Ezt a komplex logikát nem egyetlen monolitikus osztályban, hanem több, egymásra épülő szolgáltatás (DailyNoteService, MealEntriesService, MealFoodsService) együttműködésével valósítottam meg. Ez a felépítés biztosítja a rendszer modularitását és a felelősségi körök tiszta szétválasztását.

4.2.7.1 DailyNote életciklusa (Inicializálás)

A naplózási folyamat alapja, hogy létezzen az adott napra vonatkozó keretrendszer. Ezt a DailyNoteService vezérli. Amikor a felhasználó belép a napló felületre, a rendszer ellenőrzi a napi bejegyzés meglétét. Hiány esetén a CreateDailyNote metódus automatikusan létrehozza azt, és ami a legfontosabb: ebben a pillanatban rögzíti a felhasználó aktuális céljait (a UserProfile-ból átmásolja a TargetCalorie, TargetProtein stb. értékeket). Ez a tervezési döntés biztosítja az adatbiztonságot, ha a felhasználó később módosítja a profilját, a múltbeli naplói nem torzulnak, azok megőrzik az akkori célértékeket.

4.2.7.2 Étkezések hierarchiája (MealEntries)

A naplón belüli étkezések (pl. Reggeli, Ebéd) kezelését a MealEntriesService végzi. A rendszert úgy terveztem, hogy egy DailyNote tetszőleges számú étkezést tartalmazhat, amelyeket a MealType adatok (pl. "Reggeli", "Vacsora") alapján kategorizálok. Ez a réteg felel az adott napszak összesített tápértékeinek (SumCalorie, SumProtein) tárolásáért, amelyeket a rendszer valós időben frissít, amint a felhasználó új tételt ad hozzá.

4.2.7.3 Tételek rögzítése és a számítási lánc (Recalculation Chain)

A legalsó szinten a konkrét ételek és receptek hozzáadása történik, amit a MealFoodsService és MealRecipesService kezelnek. A legnagyobb kihívást itt az adatok konzisztenciájának megőrzése jelentette minden egyes módosítás után.

Erre egy többlépcsős újraszámolási láncot (Recalculation Chain) alkalmaztam:

A felhasználó hozzáad egy ételt (pl. 200g csirkemell). A MealFoodsService elmenti a tételt, majd szolgáltatás azonnal lekéri az érintett étkezéshez (MealEntries) tartozó összes ételt és receptet, majd újrakalkulálja az étkezés tápértékeit (RecalculateMealEntryNutrition). A lánc végén a rendszer meghívja a DailyNoteService-t, amely összegzi az összes étkezés adatát, és frissíti a napi főösszegeket.

Ez a hierarchikus működés garantálja, hogy a felhasználó a felületen (pl. a folyamatsávokon) mindig matematikai pontossággal a helyes, aktuális értékeket lássa, függetlenül attól, hogy egyszerű ételt vagy összetett receptet rögzített.

4.2.7.4 Testsúly naplózása

A testsúlyváltozás követése lényeges funkció, a célok elérése érdekében, amit a DailyNoteService UpdateWeight-tel valósítottam meg. A rendszer az aktuális napi naplóbejegyzésben (DailyNote) tárolja el a mért súlyt. Ez teszi lehetővé a későbbi történeti visszatekintést és a grafikonos ábrázolást. A következő napi testsúly az itt megadott adattal fog inicializálódni.

4.2.7.5 Sporttevékenységek kezelése

A rendszer nemcsak a bevitelt, hanem az energiafelhasználást is nyomon követi. A sporttevékenységek rögzítését és kalkulációját a UserActivityService osztályban valósítottam meg. A folyamat a felhasználó által megadott időtartam és a kiválasztott mozgásforma alapján zajlik.

A első lépésként a rendszer lekéri a választott mozgásformához tartozó katalógusadatokat (ActivityCatalog), amely tartalmazza az egységnyi időre (pl. 60 perc) eső kalóriaégetést. A rendszer automatikusan elvégzi a számítást: a katalógusértéket arányosítja a felhasználó által megadott időtartammal (Duration). A mentést követően a szolgáltatás azonnal meghívja a DailyNoteService-t, hogy frissítse az adott naphoz tartozó összesített „elégetett kalória” mezőt, így a felhasználó valós időben látja az energiamérlegének változását.

4.2.8 Adatszolgáltatás a vizualizációhoz (Grafikon és Naptár)

A frontend oldali vizualizáció (súlygrafikon, naptárnézet) kiszolgálása érdekében a backend specifikus végpontokat biztosít, amelyek adatokat szolgáltatnak.

- Naptárnézet: A felhasználó itt tudja kiválasztani mely napi bejegyzését szeretné megtekinteni.
- Súlygrafikon: Időrendben adja vissza a rögzített testsúly adatokat. Mivel a DailyNote tartalmazza a DailyWeight mezőt és a CreatedAt dátumot, egy egyszerű LINQ lekérdezéssel kinyerhető a felhasználó súlyváltozásának teljes története, amit a frontend közvetlenül a diagram kirajzolásához használ.

4.2.9 API Dokumentáció

A modern webfejlesztésben elengedhetetlen az API megfelelő dokumentálása. Erre a célra a Swagger/OpenAPI szabványt használtam, amelyet az NSwag eszköztárral egészítettem ki.

A Program.cs-ben konfiguráltam a Swagger generátort, amely futásidőben feltérképezi a Controllereket és a DTO-kat, majd létrehoz egy interaktív dokumentációs felületet (/swagger).

Ennek a megoldásnak a legnagyobb előnye a frontend fejlesztés során jelentkezett. Az NSwag alkalmazás segítségével egyetlen kattintással képes voltam legenerálni a teljes TypeScript kliens kódot (NswagGenerated.ts). Ez azt jelentette, hogy a frontend oldalon nem kellett manuálisan megírnom a HTTP hívásokat és a modelleket, hanem egy típusos támogatott szolgáltatást kaptam, ami jelentősen csökkentette a hibalehetőségeket és felgyorsította a munkát (Az alkalmazás beállítási egy külön fájlban tárolódnak amire érdemes vigyázni. Esetemben előfordult, hogy fejlesztés során kitöröltem ezt a fájlt és órákat töltöttem azzal, hogy újra rájöjjenek mi volt a helyes beállítás.)

4.3 A Frontend megvalósítása

A felhasználói felületet Angular 19 keretrendszer segítségével, modern egyoldalas alkalmazásként (SPA) valósítottam meg. A kliens oldali fejlesztés során a fő célom egy reszponzív, gyors és a backenddel szorosan integrált felület kialakítása volt. Ebben az alfejezetben bemutatom a kliens architektúráját, a backenddel való kommunikáció módját, valamint a legösszetettebb komponensek belső logikáját.

4.3.1 Kliens oldali architektúra és modularitás

Az Angular alkalmazásomat tudatosan moduláris architektúrára terveztem, hogy a kód átlátható, karbantartható és logikusan strukturált legyen. A fejlesztés során a „Separation of Concerns” (Felelősségi körök szétválasztása) elvét követtem, így az alkalmazás különböző funkcionális egységeit dedikált könyvtárakba és modulokba szerveztem a src/app mappán belül.

4.3.1.1 Moduláris felépítés

A monolitikus struktúra helyett (ahol minden egyetlen modulban van) három fő modulra bontottam az alkalmazást:

AppModule (Fő belépési pont): Ez a gyökérmodul (`app.module.ts`), amely összefogja az alkalmazást. Itt importálom a globális függőségeket (pl. `BrowserModule`, `HttpClientModule`). Ez a modul felelős az alkalmazás elindításáért és az alapvető navigáció kezeléséért.

AccountModule (Felhasználókezelés): A felhasználói azonosítással és fiókkezeléssel kapcsolatos összes logikát egy dedikált modulba (`AccountModule`) szerveztem ki. Ide tartoznak a bejelentkezés (`LoginComponent`), a regisztráció (`RegisterComponent`), valamint a jelszó visszaállítás komponensei. Ezzel a megoldással elértem, hogy a hitelesítési logika fizikailag is elkülönüljön az alkalmazás üzleti funkcióitól, ami átláthatóbbá tette a kódbázist.

SharedModule (Megosztott erőforrások): A kódDuplikáció elkerülése (DRY - Don't Repeat Yourself elv) érdekében létrehoztam egy `SharedModule`-t. Ide gyűjtöttem minden olyan komponenst, amelyet az alkalmazás több pontján is felhasználok.

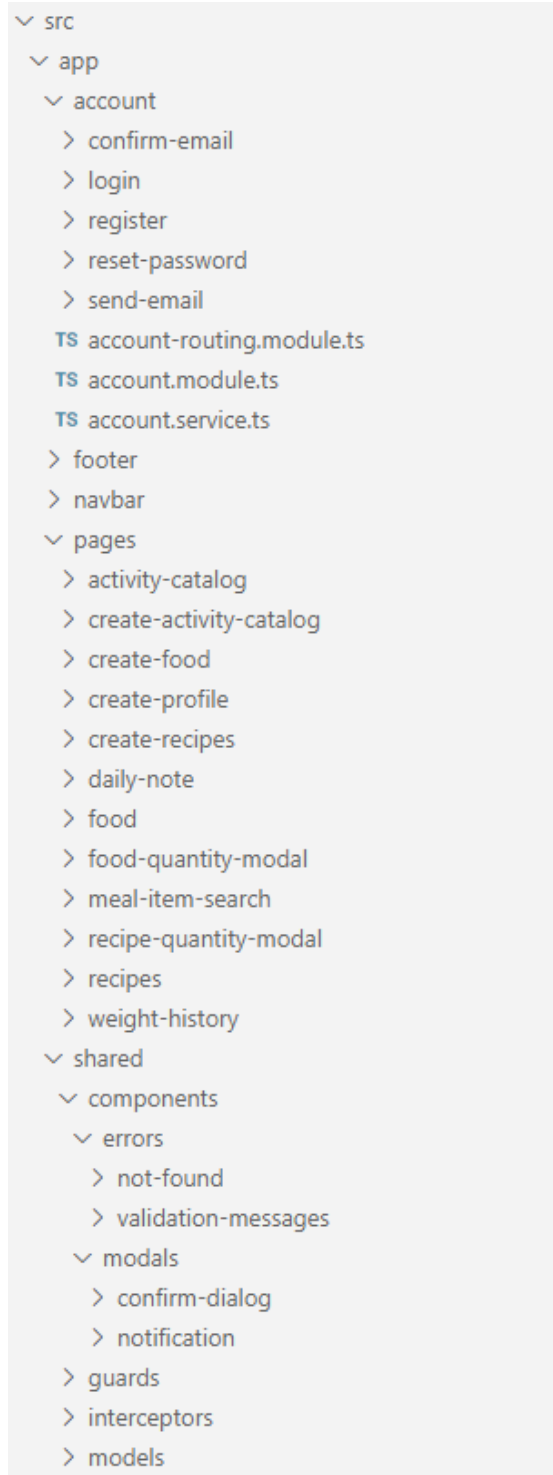
- Itt deklaráltam a `ValidationMessagesComponent`-et, amely egységesen jeleníti meg az űrlaphibákat.
- Itt deklaráltam a `ConfirmDialogComponent`-et, amelyet egységesen használtam az elemek törlésének megerősítésére.
- Ide került a `NotificationComponent` is, amely a felugró visszajelzéseket kezeli. Ennek a modulnak a használata lehetővé tette, hogy ezeket a komponenseket bárhol egyszerűen beilleszthessem, anélkül, hogy újra kellene definiálnom őket.

4.3.1.2 Könyvtárstruktúra

A fájlrendszer szintjén is szétválasztottam az üzleti és a technikai logikát:

- **pages könyvtár:** Ide helyeztem az alkalmazás "érdemi" oldalait, ahol a felhasználó a napi tevékenységét végzi. Itt találhatóak a naplózás (`daily-note`), a receptkezelés (`recipes`, `create-recipes`) és a profilkezelés (`create-profile`), étel kezelés (`food`, `create-food`), aktivitás kezelés (`activity-catalog`, `create-activity-catalog`) és az ezek működéséhez szükséges egyéb komponensek.

- shared könyvtár: Ez tartalmazza a technikai infrastruktúrát: a modelleket (DTO-k), a Guard-okat (útvonalvédelem), az Interceptorokat (token kezelés) és az újrafelhasználható UI elemeket.



6. ábra Könyvtárstruktúra

4.3.1.3 Az alkalmazás kerete (Layout)

A felhasználói felület vázát és az alkalmazás belépési pontját az AppComponent adja. Ennek implementálásakor három fő technikai kihívást kellett megoldanom: a reszponzív, mindig az oldal alján elhelyezkedő lábléct (Footer), a navigáció reaktív állapotkezelését, valamint a felhasználói élmény érdekében az aktuális friss adatok megjelenítése az oldal újratöltésekor.

Layout technikai megvalósítása (Footer): A webfejlesztésben gyakori probléma, hogy kevés tartalom esetén a lábléc "felcsúszik" a képernyő közepére. Ezt a problémát modern CSS Flexbox technikával oldottam meg az app.component.html fájlban. A body elemet flexibilis konténerként definiáltam, amelynek minimális magasságát a képernyő teljes magasságára (min-height: 100vh) állítottam, az elemek irányát pedig oszloposra (flex-direction: column).

A struktúra három fő elemből áll:

- app-navbar: A felső navigációs sáv fix magassággal.
- Tartalomtároló (div.container): Ennek az elemnek a flex: 1 stílust adtam. Ez a beállítás arra utasítja a böngészőt, hogy ez az elem töltsen ki az összes rendelkezésre álló szabad helyet a fejléc és a lábléc között. Így, ha a tartalom kevés, a konténer akkor is "kinyúlik", és a lábléct a képernyő aljára szorítja.
- app-footer: A lábléc, amely így mindig vizuálisan a helyén marad. A FooterComponent-ben dinamikus dátumkezelést is implementáltam (new Date().getFullYear()), hogy a copyright évszám mindig aktuális legyen.

Reaktív Navigáció és Menükezelés: A menüpontok láthatóságát (pl. "Login" vs. "Logout") nem manuális változókkal, hanem reaktív módon, az Angular AsyncPipe (| async) segítségével vezérelem. A nézet közvetlenül feliratkozik az AccountService-ben található user\$ observable-re.

- Ha az érték null, a rendszer a vendég-menüt (Bejelentkezés, Regisztráció) rendereli ki.
- Ha van érvényes User objektum, a rendszer a bejelentkezett nézetet mutatja, megjelenítve a felhasználó keresztnévét és a "Logout" gombot.

A felhasználói élmény javítása érdekében a `routerLinkActive="active"` direktívát használtam. Ez automatikusan hozzáadja az `active` CSS osztályt ahhoz a menüponthoz, amelyen a felhasználó éppen tartózkodik, vizuális visszajelzést adva a navigációról. Esetemben beállítottam, hogy az éppen aktuális oldalnak gombját más színnel jelezze.

Alkalmazás inicializálása és Token-ellenőrzés: Az egyoldalas alkalmazások kritikus pontja, hogy az oldal frissítésekor (F5) ne vesszen el a bejelentkezett állapot. Ezt a logikát az `AppComponent`-ben, az `ngOnInit` valósítottam meg a `refreshUser()` metódussal.

Az alkalmazás indulásakor a rendszer ellenőrzi, van-e elmentett JWT token a böngésző `localStorage`-ában (`this.accountService.getJWT()`). Ha talál token, egy API hívást indít a backend felé (`refreshUser`), hogy ellenőrizze annak érvényességét és lekérje a friss felhasználói adatokat.

Külön figyelmet fordítottam a lejárt tokenek kezelésére. A `subscribe` blokk `error` ágában figyelem a `401 Unauthorized` státuszkódot. Ha a szerver elutasítja a token (pl. lejárt), a kliens automatikusan kijelentkezteti a felhasználót (`logout`), és a `SharedService`-en keresztül egy felugró értesítésben ("Account blocked") tájékoztatja a hiba okáról.

4.3.2 Navigáció és útvonal védelem (Routing & Security)

Egy egyoldalas alkalmazásnál (SPA) kritikus fontosságú a kliens oldali útvonalválasztás (Routing) helyes konfigurálása, hiszen ez határozza meg, hogy a felhasználó milyen tartalmat lát az URL alapján, anélkül, hogy az oldal újra töltődne. Ezt a logikát az Angular beépített `RouterModule`-jára építve, az `app-routing.module.ts` fájlban definiáltam.

4.3.2.1 Útvonalak definíciója és hierarchiája

A routes tömb összeállításakor logikailag több fő csoportba rendeztem az elérési útvonalakat, amelyek különböző hozzáférési szinteket és technikai megoldásokat igényeltek.

Publikus útvonalak (Public Routes): Azok az oldalakat, amelyek bejelentkezés nélkül is elérhetők bárki számára. Úgy terveztem a rendszert, hogy a látogatók

regisztráció nélkül is megtekinthessék az adatbázis tartalmát (katalógusokat), de módosítani ne tudjanak.

- path: 'food': Az ételek listázása (FoodComponent).
- path: 'activitycatalog': A sporttevékenységek katalógusa (ActivityCatalogComponent).
- path: 'recipes': A receptek listája (RecipesComponent).

Beállítottam egy alapértelmezett átirányítást is: ha a felhasználó az üres gyökér útvonalra (path: "") érkezik, a rendszer automatikusan a /food oldalra navigálja, így a belépéskor azonnal releváns tartalmat lát.

Az alkalmazásom egyik legfontosabb architektúrális döntése a védett útvonalak csoportosítása volt. Ahelyett, hogy minden egyes szerkesztő felülethez (Create/Edit) külön külön rendeltem volna hozzá a jogosultság ellenőrzést, egyetlen közös szülő útvonalat hoztam létre.

Ezt a szülő útvonalat üres útvonalként (path: "") definiáltam, és ehhez rendeltem hozzá a canActivate: [AuthorizationGuard] védelmet. A tényleges funkcionális oldalakat ennek a szülőnek a children (gyermek) tömbjében helyeztem el.


```

{ path: '',
  runGuardsAndResolvers:'always',
  canActivate:[AuthorizationGuard],
  children:[
    { path: 'activitycatalog/create', component: CreateActivityCatalogComponent },
    {path: 'activitycatalog/edit/:id', component: CreateActivityCatalogComponent },
    { path: 'food/create', component: CreateFoodComponent },
    { path: 'food/edit/:id', component: CreateFoodComponent },
    { path: 'recipes/create', component: CreateRecipesComponent },
    { path: 'recipes/edit/:id', component: CreateRecipesComponent },
    { path: 'create-profile', component: CreateProfileComponent },
    { path: 'dailynote', component: DailyNoteComponent },,],},

{ path: '',
  runGuardsAndResolvers:'always',
  canActivate:[AuthorizationGuard],
  children:[
    { path: 'activitycatalog/create', component: CreateActivityCatalogComponent },
    { path: 'activitycatalog/edit/:id', component: CreateActivityCatalogComponent },
    { path: 'food/create', component: CreateFoodComponent },
    { path: 'food/edit/:id', component: CreateFoodComponent },
    { path: 'recipes/create', component: CreateRecipesComponent },
    { path: 'recipes/edit/:id', component: CreateRecipesComponent },
    { path: 'create-profile', component: CreateProfileComponent },
    { path: 'dailynote', component: DailyNoteComponent },,],},

```

Ennek a hierarchikus felépítésnek kettős előnye van:

- Öröklődés: A children tömbben lévő összes útvonal (pl. /dailynote, /recipes/create) automatikusan öröklí a szülőre tett Guard-ot, így garantált, hogy a naplóhoz vagy a profil szerkesztéséhez (create-profile) csak bejelentkezett felhasználó férhet hozzá.
- Dinamikus paraméterek: A szerkesztő útvonalaknál (pl. recipes/edit/:id) útvonal-paramétereket (:id) alkalmaztam. Ezt az azonosítót a komponensekben az ActivatedRoute segítségével olvasom ki, hogy a backendről betölthessem a szerkesztendő elem adatait.

Lusta betöltés (Lazy Loading): Az alkalmazás indulási idejének optimalizálása érdekében a felhasználókezelést (AccountModule) leválasztottam. A path: 'account' útvonalnál a loadChildren szintaxist alkalmaztam. Ez azt eredményezi, hogy a bejelentkezéshez és regisztrációhoz szükséges kód nem töltődik le az első oldalbetöltéskor, hanem csak akkor, amikor a felhasználó ténylegesen rákattint a "Login" gombra.

Végezetül a hibás URL-ek kezelésére a „wildcard” (**) útvonalat használtam, amely minden, a fentiekben nem definiált kérést a NotFoundComponent-re (404-es hibaoldal) irányít át.

4.3.2.2 Útvonal védelem (AuthGuard) megvalósítása

A jogosulatlan hozzáféréseket az AuthorizationGuard osztállyal akadályoztam meg (shared/guards/authorization.guard.ts), amely a CanActivate interfészt valósítja meg. A Guard feliratkozik az AccountService-ben található user\$ observable-re, amely a felhasználó aktuális állapotát (be van-e jelentkezve) jelzi. A map operátor segítségével megvizsgálom a user objektumot. Ha létezik (nem null): A felhasználó hitelesítve van, a Guard true értékkel tér vissza, engedélyezve a navigációt. Ha nem létezik: A rendszer azonnal megakadályozza a tovább haladást (false). Ezzel egyidőben a SharedService segítségével egy felugró értesítésben ("Restricted Area") tájékoztatom a felhasználót, és átirányítom a bejelentkezési oldalra.

4.3.3 Kommunikáció a backenddel (API Layer)

A modern webalkalmazások egyik legkritikusabb pontja a kliens és a szerver közötti adatcsere. A fejlesztés során a fő célkitűzésem az volt, hogy ez a kommunikáció típusbiztos, automatizált és könnyen karbantartható legyen, minimalizálva a hiba lehetőségét.

4.3.3.1 Automatizált kliensgenerálás (NSwag)

A fejlesztés során a hagyományos, manuális megközelítés helyett (ahol minden kérést kézzel írok meg) az automatizációt választottam. A backend oldalon konfigurált OpenAPI/Swagger specifikációt használtam fel arra, hogy az NSwag eszköztár segítségével legeneráljam a teljes kliens oldali adatelérési réteget.

Az NSwag minden egyes backend kontrollerhez létrehozott egy megfelelő Angular Injectable-t. Ezek az osztályok tartalmazzák a végpontok hívásához szükséges teljes logikát (URL összeállítás, HTTP metódusok, hibakezelés). A projektben számos kliensosztály jött létre, amelyek közül a legfontosabbak:

- AccountClient: A felhasználókezelésért felel (pl. login, register, confirmEmail metódusok), közvetlen kapcsolatot biztosítva az AccountController-rel.

- `DailyNoteClient`: A naplózás központi eleme. Metódusai (pl. `getDailyNote`, `addFoodToMeal`) teszik lehetővé a napi bejegyzések lekérdezését és módosítását.
- `RecipesClient` és `FoodClient`: Az ételek és receptek kezelését végzik (pl. `createRecipe`, `searchFood`), biztosítva a CRUD műveleteket.
- `UserActivityClient` és `ActivityCatalogClient`: A sporttevékenységek naplózását és a katalógus elérését támogató szolgáltatások.
- `MealEntriesClient`, `MealFoodsClient`: A napló részleteinek (étkezések, tételek) kalkulását végző kliensek.

Ezeket a klienseket a komponenseimben (pl. `DailyNoteComponent`) függősinjektálással (DI) érem el, így a kódom tiszta és típusos marad.

A kliensfájl nemcsak a logikát, hanem a teljes adatmodellt is tartalmazza TypeScript interfészek formájában. Ez garantálja, hogy a frontend és a backend egy nyelvet beszéljen. A generált modellek között megtalálhatóak:

Válasz modellek (Response DTOs): Mint például a `DailyNoteResponseDto`, `UserProfileResponseDto` vagy `RecipeResponseDto`. Ezek pontosan definiálják, milyen adatokat kap a kliens a szervertől (pl. a `DailyNoteResponseDto` tartalmazza a `dailyTargetCalorie` és `actualCalorie` mezőket).

Létrehozási modellek (Create DTOs): Az adatok küldésekor használt struktúrák, mint a `RecipeCreateDto` vagy `UserActivityCreateDto`. Ezek biztosítják, hogy csak valid struktúrájú adatot küldhessek a szervernek.

Segédtypusok: Olyan típusok és interfészek (pl. `MealTypeResponseDto`, `FoodResponseDto`), amelyek a komplexebb objektumok kezeléséhez használtam.

4.3.3.2 Token kezelés (`HttpInterceptor`)

A védett API végpontok eléréséhez minden egyes HTTP kérés fejlécében küldeni kell az érvényes JWT token. A DRY (Don't Repeat Yourself) elvét követve ezt a logikát nem a komponensekbe építettem be, hanem egy `HttpInterceptor` segítségével központosítottam.

Implementáltam a `JwtInterceptor` osztályt, amely implementálja az Angular `HttpInterceptor` interfészét. Ez a technológia "middleware-ként" működik a kliens oldalon: minden kifelé menő kérést elfog, módosít, majd továbbít.

Az `intercept` metódusban feliratkozom az `AccountService`-ben tárolt `user$` observable-re, de a `take(1)` operátorral biztosítom, hogy csak az aktuális értéket vegye ki, és azonnal le is iratkozzon (így elkerülhető a memóriaszivárgás). Megvizsgálom, hogy a felhasználó be van-e jelentkezve (létezik-e a `user` objektum és a hozzá tartozó `jwt` token). Mivel az Angularban a `HttpRequest` objektumok immutábilisak (nem módosíthatók), a kérést a `.clone()` metódussal lemásolom. A másolat fejlécéhez adom hozzá az `Authorization` kulcsot a `Bearer <token>` értékkel. A módosított (klónozott) kérést adom át a `next.handle()` metódusnak, amely ténylegesen elküldi azt a szerver felé.

```

    intercept(request: HttpRequest<unknown>, next: HttpHandler):
    Observable<HttpEvent<unknown>> {
      this.accountService.user$.pipe(take(1)).subscribe({
        next: user => {
          if (user) {
            request = request.clone({
              setHeaders: {
                Authorization: `Bearer ${user.jwt}`
              }
            });
          }
        }
      });
    }
  }
}
return next.handle(request);
}
}

```

4.3.4 Felhasználókezelés (Account Module)

A felhasználói fiókok kezelését, a hitelesítést és a biztonsági funkciókat egy modulban, az AccountModule-ban valósítottam meg. Ez a modul fizikailag is elkülönül az alkalmazás többi részétől (src/app/account), és saját útvonalválasztóval (AccountRoutingModule) rendelkezik, amely a lusta betöltés (Lazy Loading) révén csak szükség esetén töltődik be a böngészőbe.

4.3.4.1 Reaktív Állapotkezelés (AccountService)

A felhasználókezelés "agya" az AccountService osztály. Itt valósítottam meg azt a logikát, amely biztosítja, hogy az alkalmazás mindig tisztában legyen azzal, ki van bejelentkezve.

A legnagyobb kihívást az jelentette, hogy az Angular komponensek (pl. a Navbar) aszinkron módon töltődnek be, így biztosítanom kellett, hogy a bejelentkezési állapot bárki számára, bármikor elérhető legyen. Erre a célra egy ReplaySubject<User | null>(1) típusú objektumot (userSource) alkalmaztam. Szemben a hagyományos Subject vagy BehaviorSubject megoldásokkal, a ReplaySubject(1) eltárolja az utolsó kibocsátott értéket az új feliratkozóknak, így, ha az AuthGuard csak később iratkozik fel, akkor is azonnal megkapja az aktuális felhasználói objektumot.

Az oldal újratöltésekor (F5) a munkamenet folytonosságát a localStorage-ba mentett adatokkal és a refreshUser metódussal biztosítom, amely automatikusan megújítja az állapotot, ha talál érvényes tokent.

4.3.4.2 Regisztráció és bejelentkezés

A oldalak kezeléséhez minden esetben az Angular Reactive Forms modulját használtam, mivel ez lehetőséget ad a komplex validációs logika (pl. jelszóerősség) TypeScript oldali megvalósítására és tesztelésére. Az oldalak (registerForm) validátorai biztosítják, hogy a név minimum 3 karakter legyen, az e-mail cím megfeleljen a regexnek, a jelszó pedig 6-15 karakter hosszú legyen. A "Create Account" gomb megnyomásakor a komponens feliratkozik az `AccountService.register()` metódusára. Siker esetén a `SharedService` segítségével értesítem a felhasználót, majd a bejelentkezési oldalra navigálok.

A `LoginComponent` nemcsak a hitelesítést végzi, hanem kezeli a felhasználói utat és a hibákat is. A komponens inicializálásakor (`ngOnInit`) kiolvasom a `returnUrl` paramétert az URL-ből. Ha a felhasználó korábban egy védett oldalról (pl. `/dailynote`) lett átirányítva a bejelentkezéshez, sikeres belépés után a rendszer automatikusan visszavigyíti oda. A oldalakon külön figyelmet fordítottam arra az esetre, ha a bejelentkezés azért hiúsul meg, mert a felhasználó még nem erősítette meg az e-mail címét. Ha a hibaüzenet tartalmazza a "Please confirm your email" szöveget, megjelenítek egy linket ("Click here to resend email confirmation link..."), amely meghívja a `resendEmailConfirmationLink()` metódust.

4.3.4.3 Elfelejtett jelszó és E-mail megerősítés

A `ConfirmEmailComponent` komponens végzi a regisztráció véglegesítését. Az `ngOnInit` életciklusban feliratkozom az `ActivatedRoute` paramétereire, és kinyerem az URL-ből érkező hitelesítő tokent és e-mail címet. Ezekkel az adatokkal automatikusan meghívom az `AccountService.confirmEmail` metódust. Siker esetén (`success = true`) a felhasználó egy "Login" gombot lát, hiba esetén (pl. lejárt token) pedig a rendszer felajánlja a megerősítő link újraküldését a `resendEmailConfirmationLink()` metódus segítségével.

A `SendEmailComponent` kód duplikáció elkerülése érdekében egyetlen komponenszt készítettem az "Elfelejtett jelszó" és az "E-mail megerősítés újraküldése" funkciókhoz. A komponens az URL paraméter alapján dönti el, melyik üzemmódban működjön.

A ResetPasswordComponent felület fogadja a felhasználót, amikor az e-mailben kapott jelszó helyreállító linkre kattint. A rendszer automatikusan beilleszti a token-t és az e-mail címet az űrlapba (az e-mail mezőt letiltva), így a felhasználónak csak az új jelszót kell megadnia a resetPasswordForm-ban.

4.3.5 Adatkezelés és törzsadatok (CRUD)

A naplózási funkció működéséhez elengedhetetlenek a törzsadatok. Megvalósítottam a Food (Ételek) és ActivityCatalog (Tevékenységek) entitások teljes körű kezelését (Létrehozás, Listázás, Szerkesztés, Törlés). Mivel ezek összetett felületek, nem modális ablakokat, hanem dedikált oldalakat használtam.

A FoodComponent és az ActivityCatalogComponent felelős az adatok megjelenítéséért. A loadFoods() és loadActivities() metódusok az inicializáláskor (ngOnInit) töltik be az adatokat. A törlési logika (onDelete) biztonsági okokból egy megerősítő ablakot dob fel (amit a ConfirmDialogComponent-el oldottam meg, hogy újra felhasználható legyen). Jóváhagyás esetén a komponens meghívja a törlő végpontot, majd siker esetén, kliens oldalon, a filter metódussal távolítja el az elemet a listából, így nem szükséges újratölteni az egész táblázatot a szerverről.

A kódDuplikáció elkerülése érdekében ugyanazt azt a komponens-t (CreateFoodComponent, CreateActivityCatalogComponent) használok létrehozásra és szerkesztésre is. Az ngOnInit-ben vizsgálom az útvonal paramétereit (route.paramMap). Ha van ID, a komponens Szerkesztés módba vált (editing = true) és betölti az adatokat. A createOrUpdate metódus dönti el az állapot alapján, hogy a create (POST) vagy az update (PUT) szolgáltatást hívja meg.

Food Edit

Title

Krumpli

Protein (g)

10

Fat (g)

10

Carb (g)

7

Calorie

103

Gram

100

Save

7. ábra Food szerkesztési felület

4.3.5.1 Listázás és ui szintű jogosultságkezelés

A FoodComponent és az ActivityCatalogComponent felelős az adatok táblázatos vagy kártyás megjelenítéséért. Tekintettel arra, hogy a rendszer specifikációja szerint a vendég felhasználók is megtekinthetik az adatokat, de módosítani csak a regisztrált tagok tudják, egy nézet-szintű védelmet alkalmaztam a HTML sablonokban. Az Angular *ngIf strukturális direktívájával és az AsyncPipe segítségével közvetlenül feliratkoztam az AccountService.user\$ observable-re.

```
<div *ngIf="(_accountService.user$ | async) as user">
  <button class="btn btn-sm btn-primary"
    (click)="onEdit(food)">Edit</button>
  <button class="btn btn-sm btn-danger"
    (click)="onDelete(food)">Delete</button>
</div>
```


Ez a megoldás biztosítja, hogy a szerkesztő és törlő gombok fizikailag le sem renderelődnek a DOM-ban, ha a felhasználó nincs bejelentkezve, így a felület letisztult marad a vendégek számára is.

A törlés kritikus művelet, ezért a delete metódus meghívása előtt egy natív megerősítő ablakot jelenítek meg. Ha a felhasználó jóváhagyja a műveletet, akkor sikeres válasz esetén nem töltöm újra a teljes listát a szerverről, hanem kliens oldalon, a filter metódussal távolítom el a törölt elemet a foods tömbből, ezzel csökkentve a hálózati forgalmat és javítva a válaszütemet.

4.3.6 Komplex űrlapkezelés: receptkészítő

Az alkalmazás egyik legösszetettebb űrlapja a CreateRecipesComponent, amely lehetővé teszi, hogy a felhasználó meglévő alapanyagokból állítson össze új ételeket. A recept létrehozása előtt a felhasználó dinamikusan adhat hozzá, módosíthat és törölhet hozzávalókat.

4.3.6.1 Hozzávalók dinamikus kezelése

A recept összetevőit hanem egy lokális tömbben, ingredients néven tárolom (RecipeFoodItemDto[]). A felhasználói interakciót a következő logikai lépésekre bontottam: A komponens betöltésekor (ngOnInit) a loadFoods() metódus feltölti a foodOptions listát. A felhasználó ebből a legördülő menüből választ alapanyagot, és megadja a mennyiséget grammal. A listában való tájékozódást a getFoodTitleById(id) segédfüggvény segítségével oldottam meg, amely az ID alapján megjeleníti az étel nevét.

Hozzáadás vagy Frissítés esetében (addOrUpdateIngredient) gombnyomásra a rendszer ellenőrzi, hogy szerkesztésről vagy új hozzáadásról van-e szó. Új elem esetén a push() metódussal adom a tömbhöz. Meglévő elem szerkesztésekor (editingIngredientIndex nem null) közvetlenül a tömb adott indexű elemét frissítem.

Módosítás és Törlés esetén a lista minden soránál elhelyeztem egy "Edit" és "Remove" gombot. A loadIngredientForEdit(index) visszatölti az adatokat a beviteli mezőbe, lehetővé téve a javítást. A removeIngredient(index) a splice() metódussal véglegesen eltávolítja az elemet a listából.

4.3.6.2 Valós idejű számítások

A felhasználói élmény egyik kulcseleme az azonnali visszajelzés. Nem vártam meg a szerver választ a recept tápértékeinek kiszámításához, hanem az `updateNutrition()` metódust alkalmaztam.

Ez a függvény minden egyes lista módosítás (hozzáadás, törlés) után lefut. Végigiterál az `ingredients` tömbön, és a kiválasztott ételek (`foodOptions`) ismert tápértékei alapján, a mennyiségekkel súlyozva összesíti a recept teljes kalória, fehérje, zsír és szénhidráttartalmát.

```
updateNutrition(): void {
  this.sumCalorie = 0;
  this.sumProtein = 0;
  this.sumFat = 0;
  this.sumCarb = 0;

  this.ingredients.forEach(ing => {
    const food = this.foodOptions.find(f => f.foodID === ing.foodID);
    if (!food || !ing.quantity) return;

    const factor = ing.quantity / 100;
    this.sumCalorie += (food.calorie || 0) * factor;
    this.sumProtein += (food.protein || 0) * factor;
    this.sumFat += (food.fat || 0) * factor;
    this.sumCarb += (food.carb || 0) * factor;
  });
}
```

Így a felhasználó valós időben látja a receptje "alakulását" a címsorban, még a mentés előtt.

4.3.6.3 Mentés és DTO konverzió

A `createOrUpdate` metódus végzi a végső összekapcsolást. Létrehozza a `RecipeCreateDto` objektumot, amely tartalmazza a recept törzsadatait (Cím, Leírás) és a `ingredients` listát. Ezt a komplex objektumot küldöm el a backendnek, amely tranzakcionálisan menti a receptet és a hozzá tartozó kapcsolótábla bejegyzéseket.

8. ábra Recipe szerkesztési felület

4.3.6.4 Receptek megjelenítése

A RecipesComponent felelős a kész receptek prezentálásáért. Itt is érvényesül a jogosultságkezelés, bár a kártyákat bárki láthatja, a szerkesztés (/recipes/edit/:id) opció csak a bejelentkezett felhasználónak érhető el. A kártyákon már a backend által kiszámolt és tárolt összesített makróértékeket jelenítem meg, így a listázás teljesítmény igénye minimális.

4.3.7 Naplózás megvalósítása (Daily Note)

Az alkalmazás funkcionális központja a DailyNoteComponent, amely egy komplex állapotgépet valósít meg: kezeli a dátumváltást, koordinálja a modális ablakok láncolatát, és valós időben frissíti a napi összesítőket. Ebben a fejezetben részletesen bemutatom a komponens belső logikáját és az általa vezérelt folyamatokat.

4.3.7.1 Inicializálás és dátum navigáció

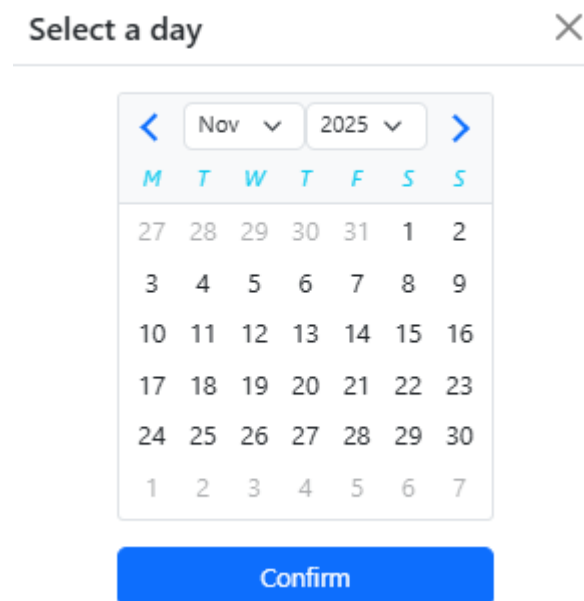
A komponens életciklusa az ngOnInit metódussal kezdődik. Itt az elsődleges feladatom az volt, hogy a rendszer képes legyen bármilyen dátum adatait betölteni, ne kizárólag az aznapit. A loadDailyNote() a központi adatbetöltő függvény. Ez a DailyNoteClient-et használva aszinkron módon lekéri a backendtől a teljes napi adathalmazt. A válasz megérkezésekor nemcsak a dailyNote változót frissítem, hanem újraszámolom a "burnedCalories" (elégetett kalória) értékét is a calculateBurnedCalories() segédfüggvény meghívásával.

A `loadPreviousNote()` és `loadNextNote()` metódusok egyszerű dátum aritmetikát végeznek, a jelenleg megjelenített dátumhoz képest egy napot vonnak ki vagy adnak hozzá, majd újra meghívják a betöltő logikát.



9. ábra Dátum léptető

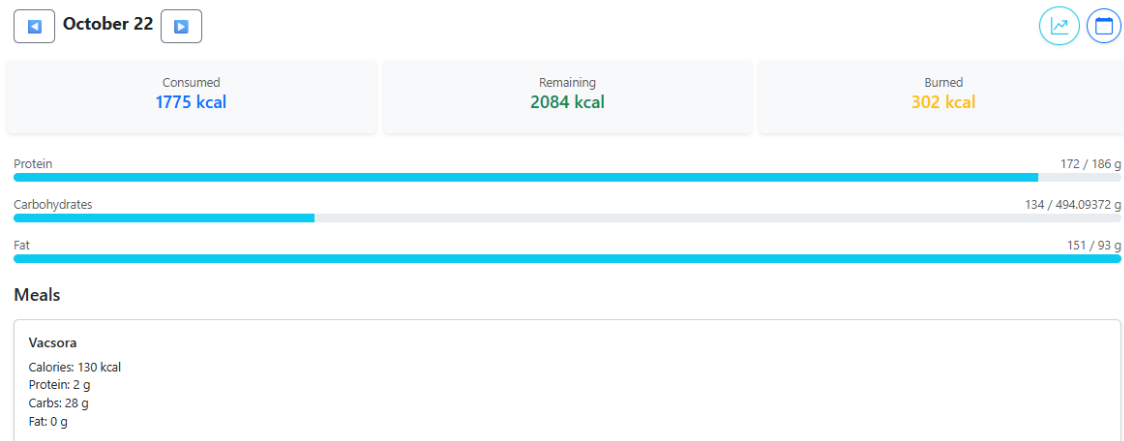
A naptár funkciót az `openCalendar()` metódus kezeli, amely egy `NgbDatepicker` komponenst tartalmazó modális ablakot nyit meg. A kiválasztás után a `handleCalendarConfirm()` végzi el a konverziót az Angular dátumformátum és a natív JavaScript `Date` objektum között, majd frissíti a nézetet.



10. ábra Naptár

4.3.7.2 Többlépcsős hozzáadási folyamat (Modal Chain)

Az új tételek rögzítését a jobb felhasználói élmény érdekében egy háromlépcsős folyamatra bontottam, ahol a `DailyNoteComponent` vezérli a modális ablakok közötti váltást.



11. ábra DailyNote részlet

1. Keresés és Szűrés (MealItemSearchComponent): A folyamat az `openMealItemSearchModal(mealEntryId)` metódussal indul, amelyet a felhasználó az "Add Food" gombra kattintva vált ki. Ez a függvény elmenti a kiválasztott étkezés (pl. Reggeli) azonosítóját, majd inicializálja a kereső komponenst. A `MealItemSearchComponent`-ben az `onInputChange()` metódus figyeli a gépelést. A `clear()` metódus biztosítja, hogy az ablak minden megnyitáskor tiszta találati listával induljon.

Vacsora Details

Calories: 130 kcal
Protein: 2 g
Carbs: 28 g
Fat: 0 g

Foods

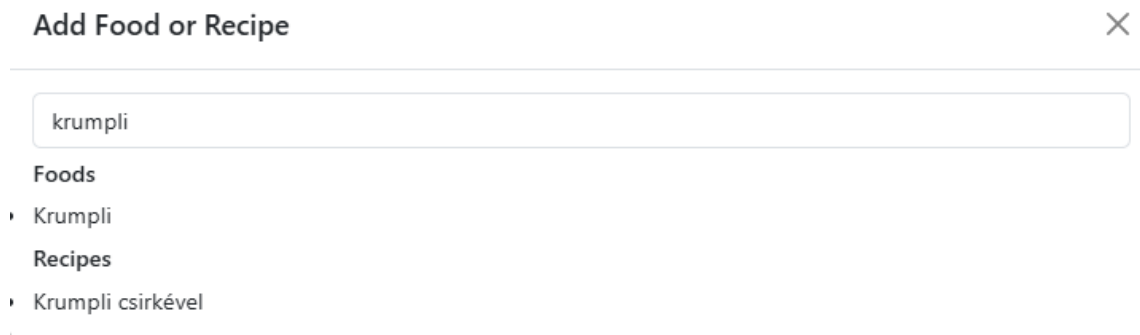
Rizs Edit Delete

Recipes

+ Add Food or Recipe

12. ábra Vacsora részletes nézet

2. Kiválasztás és Típusfelismerés: A keresőablak egy `itemSelected` eseményt használ kiválasztáskor. Ezt a fő komponens `handleItemSelected()` metódusa kapja el. Itt egy elágazással vizsgálom meg a kiválasztott elem típusát: amennyiben ételről van szó, bezárom a keresőt és a `FoodQuantityModalComponent`-et nyitom meg; recept esetén pedig a `RecipeQuantityModalComponent`-et.



Add Food or Recipe

krumpli

Foods

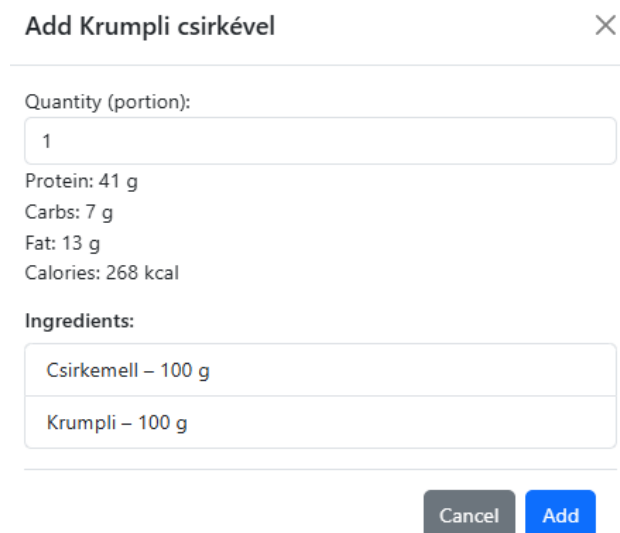
- Krumpli

Recipes

- Krumpli csirkével

13. ábra Kereső mező

3. Mennyiség megadása és Valós idejű számolás: A mennyiség-bekérő modális ablakok (`FoodQuantityModalComponent`) nemcsak adatbevitelt szolgálnak, hanem számolnak is. Implementáltam az `onQuantityChange()` metódust, amely minden billentyűleütésre (`ngModelChange`) újraszámolja a tápértékeket. Ezáltal a felhasználó azonnali visszajelzést kap arról, hogy az általa beírt mennyiség pontosan mennyi tápanyagot tartalmaz.



Add Krumpli csirkével

Quantity (portion):

1

Protein: 41 g
Carbs: 7 g
Fat: 13 g
Calories: 268 kcal

Ingredients:

Csirkemell – 100 g
Krumpli – 100 g

Cancel Add

14. ábra Mennyiség megadása

4. Mentés: A modális ablak save() metódusa végzi a tényleges API hívást. Létrehozza a megfelelő DTO-t és elküldi a szervernek. Siker esetén a modal egy added kimeneti eseményt (Output) jelez. Erre reagálva a fő komponens onItemAdded() metódusa lezárja az ablakot, eltünteti a szürke hátteret, és a loadDailyNote() meghívásával frissíti a teljes napi nézetet.

Vacsora Details X

Calories: 398 kcal
Protein: 43 g
Carbs: 35 g
Fat: 13 g

Foods

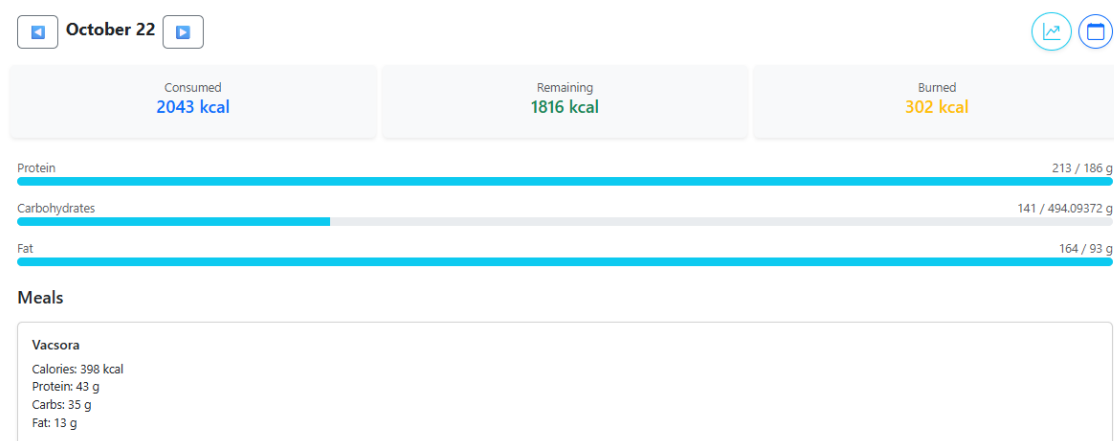
Rizs Edit Delete

Recipes

Krumpli csirkével (1 portion) Edit Delete

+ Add Food or Recipe

15. ábra Vacsora részletes nézet recept hozzáadása után



16. ábra DailyNote részlet nézet recept hozzáadása után

4.3.7.3 Szerkesztés, törlés és karbantartás

Amikor a felhasználó a listában egy tételre kattint, az editMealFood() vagy editMealRecipe() metódus fut le. Ez újra megnyitja a mennyiség-bekérő modált, de egy

fontos különbséggel: átadja neki a szerkesztendő tétel azonosítóját (`editingMealFoodId`). A modal `ngOnInit` metódusa ezt érzékeli, és "Létrehozás" helyett "Frissítés" módba kapcsol, az API hívást pedig `create`-ről `update`-re cseréli. A sporttevékenységek módosításáért az `editActivity()` metódus felel. Ez visszatölti a kiválasztott tevékenység adatait, és engedélyezi a szerkesztést.

A `deleteMealFood()` és `deleteActivity()` metódusok közvetlenül hívják a megfelelő törlő végpontokat, majd a válasz után azonnal frissítik a nézetet. Tekintettel arra, hogy az alkalmazás sok modális ablakot használ, ezért egy `closeAllModals()` technikai segédfüggvényt alkalmaztam. Ennek az a feladata, hogy ablakváltáskor vagy mentéskor manuálisan eltávolítsa a DOM-ból a beragadó modal-backdrop elemeket és a modal-open osztályt, biztosítva, hogy az alkalmazás felülete sose fagyjon le egy beragadt ablak miatt.

4.3.7.4 Testsúly, aktivitás és összesítés

A testsúly rögzítését a lehető legegyszerűbbre terveztem. Az `updateWeight()` metódust közvetlenül a beviteli mező `change` eseményéhez kötöttem, ezáltal amint a felhasználó beírja az új adatot, majd a mentésre kattint, a rendszer a háttérben automatikusan elmenti azt.

A sporttevékenységeknél a `saveActivity()` metódus menti el az időtartamot, majd ezt követően lefut a `calculateBurnedCalories()` metódus. Ez a függvény végigiterál a naphoz tartozó összes aktivitáson, és a katalógusadatok alapján kliens oldalon összegzi az elégetett kalóriákat. Ezt az értéket a grafikus felületen, a kalória-folyamatsávon (Progress Bar) egy külön színnel jelenítem meg, vizualizálva a "ledolgozott" energiát.

The screenshot displays a user interface for tracking activities and weight. At the top, there's a header 'Activities' with a '+ Add Activity' button. Below it, two activity cards are shown: 'Futás' (Running) with 'Burned: 272 kcal' and 'Duration: 30 min', and 'Programozás' (Programming) with 'Burned: 55 kcal' and 'Duration: 30 min'. Each card has a close button (X). Below the activities, there's a 'Daily Weight' section with a text input field containing '104' and a prominent blue 'Save' button.

17. ábra Aktivitás és Súly felvétel részlet

4.3.8 Profil és statisztikák

Egy egészségmegőrző alkalmazásnál kulcsfontosságú a személyre szabhatóság és a fejlődés nyomon követése. Ezt a két funkcióterületet a `CreateProfileComponent` és a `WeightHistoryComponent` segítségével valósítottam meg.

4.3.8.1 Profil varázsló és célok automatikus kezelése

A felhasználói profil nem csupán statikus adatok tárolására szolgál, hanem ez vezérli a napi kalória és makrotápanyag számításokat.

A felhasználói élmény javítása érdekében nem kérem be explicit módon, hogy a felhasználó fogyni vagy hízni szeretne-e. Ehelyett implementáltam az `updateGoalType()` metódust, amely a megadott Jelenlegi súly (`weight`) és Célsúly (`goalWeight`) mezők változásait figyeli (`valueChanges`).

Ha a célsúly kisebb, mint a jelenlegi, a rendszer automatikusan Fogyás módba áll. Ha nagyobb, akkor Tömegnövelés módba. Ha egyenlő, akkor Súlytartás a cél.

A kalóriaszükséglet (TDEE) pontosításához egy legördülő menüben kínálok fel aktivitási szorzókat (1.2-től 1.9-ig), részletes leírással (pl. "Sedentary", "Very active"), hogy a felhasználó a valósághoz legközelebb álló értéket választhassa. Az `ngOnInit` során az `AccountClient.getProfile()` segítségével próbálom betölteni a meglévő adatokat. Ha a szerver 204-es (No Content) választ küld, az azt jelenti, hogy ez egy új regisztráció, így az űrlap üresen inicializálódik. Mentéskor a `submit()` metódus validálja az űrlapot, és elküldi az `UpdateUserProfileDto`-t a backendnek.

HealthyApp

Food ActivityCalendar Recipes Daily Note

Hi Péter Logout

Edit Your Profile

First Name
péter

Last Name
pálcatori

Age
29

Height (cm)
186

Weight (kg)
93

Goal Weight (kg)
94

Body Fat (%)
15

Gender
Male

Activity Level
Moderately active (3-5 days/week)

Goal (auto-detected)
Gain weight (Bulking)

Save Profile

Copyright © 2025 HealthyApp

18. ábra Profil nézet

4.3.8.2 Adatvizualizáció: testsúlytörténet grafikon

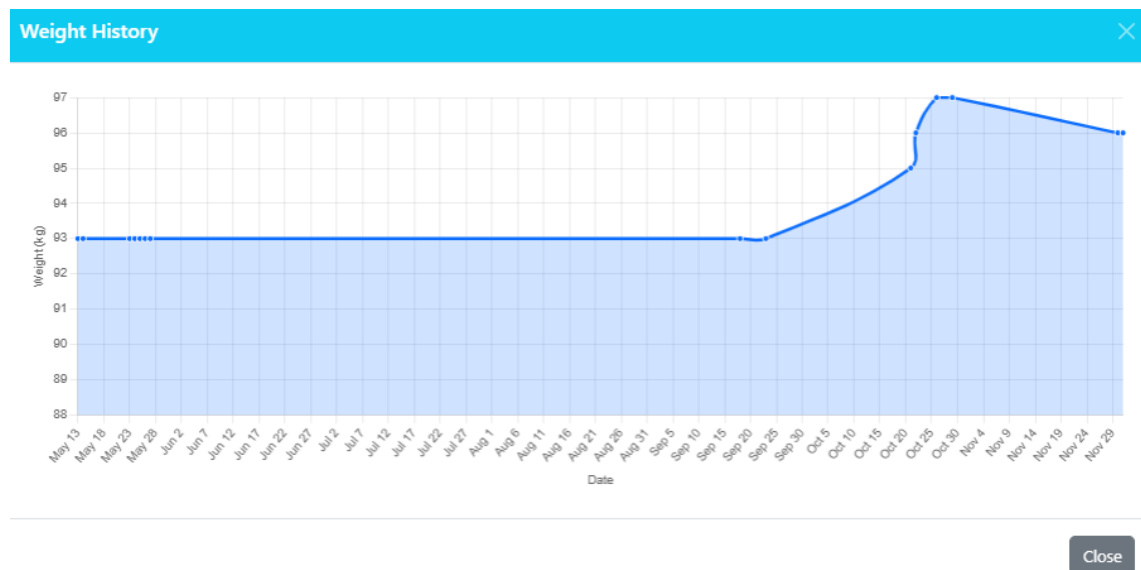
A fejlődés vizuális visszacsatolása a legmotiválóbb eszköz. A testsúly alakulásának megjelenítésére a Chart.js könyvtárat használtam a WeightHistoryComponent-be. Azért választottam a Chart.js-t, mert ez egy standard, megjelenítő, amely kiváló teljesítményt nyújt nagy adathalmazok esetén is, és rugalmasan testreszabható.

A backendtől kapott nyers DailyNoteResponseDto listát a createChartData() metódusban dolgozom fel. Kiszűröm azokat a napokat, ahol a súly 0 vagy érvénytelen, hogy ne torzuljon a grafikon. Az adatokat a Chart.js által várt { x: timestamp, y: weight } formátumra alakítom át. A grafikon csak akkor jelenik meg, ha legalább kettő érvényes mérési pont áll rendelkezésre, ellenkező esetben egy figyelmeztető üzenetet jelenítek meg.

Mivel a grafikon gyakran modális ablakban jelenik meg, kritikus volt a renderelés időzítése. Az AfterViewInit életciklusban, a ChangeDetectorRef.detectChanges() és egy rövid késleltetés (setTimeout) kombinációjával biztosítottam, hogy a <canvas> elem már biztosan létezzen a DOM-ban, mielőtt a Chart.js megpróbálna rajzolni rá. Ezzel elkerültem a modális ablakoknál gyakori "üres grafikon" hibát, miután többször szembesültem vele tesztelésnél.

A jobb olvashatóság érdekében a grafikon Y-tengelyét (Súly) nem 0-tól indítom, hanem dinamikusan számolom: a felhasználó legkisebb mért súlyából kivonok 5 kg-ot,

és ezt állítom be minimumnak. Ez biztosítja, hogy a súlyváltozás trendje (a vonal emelkedése vagy süllyedése) látványosan kirajzolódjon, még kis változások esetén is.



19. ábra Profil nézet

4.3.9 Felhasználói élmény (UX) elemek

Egy modern webalkalmazás nemcsak attól jó, hogy működik, hanem attól is, hogy folyamatosan kommunikál a felhasználóval. Éppen ezért figyeltem arra, hogy a felhasználó soha ne érezze magát elveszve. Minden sikeres vagy sikertelen műveletről visszajelzést adok neki, és kezelem a navigációs hibákat. Ezeket a funkciókat a SharedModule-ban, újrafelhasználható komponensekként valósítottam meg.

4.3.9.1 Központosított értesítési rendszer (Notification)

A rendszerüzenetek (pl. "Sikeres regisztráció", "Hiba történt") megjelenítésére nem mindig a böngésző beépített alert() ablakát, hanem az ngx-bootstrap könyvtár modális szolgáltatását (BsModalService) választottam. A NotificationComponent felelős az üzenetek megjelenítéséért, amely egy esztétikus, felugró ablakot biztosít.

A komponens logikáját úgy terveztem, hogy dinamikusan alkalmazkodjon az üzenet típusához. Egy isSuccess logikai változó vezérli a megjelenést:

- Siker esetén: A fejléc zöld háttérszín (bg-success) kap.
- Hiba esetén: A fejléc pirosra (bg-danger) vált.

Ezt a dinamikus stílusozást az Angular [ngClass] direktívájával oldottam meg a HTML sablonban. A komponenst az ngx-bootstrap BsModalRef osztályával integráltam, így az ablak bezárása közvetlenül a sablonból vezérelhető, az alkalmazás bármely pontjáról pedig programozottan megnyitható.

4.3.9.2 Újrahasznosítható validációs Üzenetek

Az űrlapok (Regisztráció, Bejelentkezés, Receptkészítés) validációjakor keletkező hibaüzenetek egységes megjelenítésére létrehoztam a ValidationMessagesComponent-et.

Ennek a komponens a nincs saját üzleti logikája, kizárólag a megjelenítésért felel. Egy @Input() dekorátorral ellátott errorMessages tömböt vár a szülő komponenstől. A HTML sablonban az *ngIf direktívával ellenőrzöm, hogy van-e megjelenítendő hiba, majd az *ngFor segítségével listázom ki azokat piros színnel. Ezzel a megoldással elértem azt, hogy nem kellett minden egyes űrlapnál újra megadnom a hibamegjelenítő HTML struktúrát, jelentősen csökkentve a kódDuplikációt.

4.3.9.3 Hibás útvonalak kezelése (404 Not Found)

A felhasználói élmény része a navigációs hibák elegáns kezelése is. Ha a felhasználó olyan URL-t ír be, amely nem létezik az alkalmazásban, a rendszer nem egy üres oldalt vagy konzolhibát mutat, hanem betölti a NotFoundComponent-et.

Ezt a komponenst a routing modul ** (wildcard) útvonalára kötöttem be. A felületen egyértelmű hibaüzenetet és egy gombot helyeztem el, amely a fő oldalra (/food) navigálja a felhasználót, így biztosítva, hogy sose kerüljön zsákutcába.

4.3.9.4 Megerősítés

A felhasználói élmény és az adatbiztonság kritikus eleme a véletlen műveletek (például törlés) megakadályozása. Nem hagytam a böngésző egyszerű window.confirm() ablakára, helyette egy saját, stílusos megerősítő ablakot (ConfirmDialogComponent) implementáltam. A komponens felépítésekor a dinamikus újrafelhasználhatóságot tartottam szem előtt. Az ngx-bootstrap BsModalRef osztályát felhasználva a modális ablak minden szöveges eleme (Cím, Üzenet, Gombok felirata) kívülről paraméterezhető, így ugyanazt a komponenst pl. használhatom étel törlésére vagy kilépés megerősítésére is. A "Yes" gomb megnyomásakor a confirm() metódus true

értéket küld és bezárja az ablakot. A "No" gomb esetén a decline() metódus false értékkel válaszol. Ez a megoldás lehetővé teszi, hogy a szülő komponens (pl. FoodComponent) feliratkozzon az eredményre, és csak pozitív visszajelzés esetén indítsa el a tényleges API hívást.

5 Önálló munka értékelése, mérések, tesztelés, eredmények bemutatása

A fejlesztési folyamat záró szakaszában elvégeztem az elkészült rendszer ellenőrzését és teljesítményértékelését. Mérnöki szempontból a legfontosabb feladatom annak igazolása volt, hogy a szoftver nemcsak a funkcionális követelményeket teljesíti, hanem architektúrális szempontból is stabil, karbantartható és hibatűrő.

5.1 Automatizált tesztelés és kódminőség

A backend üzleti logikájának validálására, automatizált tesztkörnyezetet hoztam létre a HealthyAPI.Tests projektben. Mivel az alkalmazás központi eleme a tápanyagok precíz számítása és a különböző entitások (ételek, étkezések, naplók) közötti adatkonzisztencia megőrzése, így olyan tesztek hoztam létre, hogy ne csak izolált metódusokat, hanem a Service réteg osztályainak együttműködését is vizsgáljam valóságghű körülmények között.

A tesztkörnyezet technológiai alapjait a .NET ökoszisztéma modern eszközeire helyeztem. A tesztfuttatáshoz az xUnit keretrendszert alkalmaztam. A kód olvashatóságának növelése érdekében bevezettem a FluentAssertions könyvtárat, amellyel a tesztek elvárásait természetes nyelvi formában (pl. `.Should().Be()`) fogalmaztam meg, jelentősen megkönnyítve a későbbi karbantartást. A külső függőségek, mint például a fizikai SQL Server, kiváltására az Entity Framework Core InMemory Database szolgáltatóját használtam. Ez a döntés biztosította, hogy minden tesztfuttatás egy tiszta, izolált memóriabeli adatbázison induljon el, kiküszöbölve a felesleges adatok létrehozását és a futási idő csökkenést. A felhasználói kontextus (pl. bejelentkezett User ID) szimulálására a Moq könyvtárat használtam, amellyel a http kérés viselkedését tudtam utánozni.

A tesztesetek során a „Recalculation Chain”-ra (újra számolási lánc) fókuszáltam. Részletesen vizsgáltam például a `CreateMealFoods_Should_UpdateEntrySums` esetet, ahol azt ellenőriztem, hogy egy új alapanyag hozzáadásakor a rendszer helyesen frissít-e az adott étkezés (MealEntries) összesített makróértékeit, és a változás propagálódik-e a napi napló (DailyNote) szintjére. Hasonlóan kritikus volt a `DeleteMealFoods`

forгатókönyv tesztelése, amely garantálja, hogy törlés esetén az értékek pontosan az eredeti állapotra állnak vissza. A receptkezelésnél a mennyiségi szorzók (pl. 1.5 adag étel) helyes skálázását validáltam, biztosítva, hogy a matematikai feltételeim minden esetben hibátlanul fusson le. Nem funkcionális követelmények teljesülése

A mérések és a használat során az alábbi eredményeket tapasztaltam a 3.2-es fejezetben definiált követelményekkel kapcsolatban. Az Angular SPA architektúra és a Scoped élettartamú backend szolgáltatások miatt a navigáció az oldalak között azonnali, a felhasználói élményt nem törik meg újratöltések. A Bootstrap Grid rendszer helyes alkalmazását teszteltem. A táblázatok és kártyák olvashatóan tördelődnek kis kijelzőn is. A JWT tokenek lejáratí idejének kezelése megfelelően működik; a lejárt token esetén a rendszer automatikusan kijelentkezteti a felhasználót, védve az adatokat.

5.2 Eredmények és a követelmények teljesülése

A fejlesztés eredményeként előállt szoftver sikeresen teljesítette a kitűzött célokat. A funkcionális követelmények tekintetében megvalósult a teljes körű felhasználókezelés (Identity, JWT, Mailjet), a komplex adatmodellel létrehozott étel és receptnyilvántartás, valamint a napi tevékenységek (táplálkozás, sport, testsúly) naplózása.

A nem funkcionális követelmények terén végzett méréseim alapján a rendszer teljesítménye megfelelő. Az alkalmazott Scoped élettartamú szolgáltatások és az aszinkron (async/await) adatbázis műveletek biztosítják, hogy a szerver válaszideje alacsony maradjon. A biztonsági tesztek igazolták, hogy a token-alapú hitelesítés és a szerveroldali jogosultság ellenőrzés hatékonyan védi a felhasználói adatokat az illetéktelen hozzáféréstől. A reszponzív felület (Bootstrap Grid) tesztelése során megbizonyosodtam arról, hogy az alkalmazás asztali környezetben és más is egységes felhasználói élményt nyújt.

6 Összefoglaló

Diplomamunkám keretében megterveztem és létrehoztam egy modern, többretegű webalkalmazást, amely a felhasználók egészségtudatos életmódját támogatja. A fejlesztés során sikerrel integráltam a szerveroldali ASP.NET Core és a kliensoldali Angular technológiákat, létrehozva egy reszponzív, nagy teljesítményű rendszert.

A rendszer alapjait Code-First megközelítéssel, Microsoft SQL Server adatbázison alakítottam ki, különös figyelmet fordítva a komplex (több a többhöz) kapcsolatok kezelésére. A biztonságos működést az ASP.NET Core Identity és a JWT token alapú hitelesítés, valamint a Mailjet e-mail szolgáltatás integrálásával garantáltam. A backend fejlesztésének legfontosabb pontja a „Recalculation Chain” (újraszámolási lánc) architektúra megvalósítása volt, amely biztosítja, hogy a naplóbejegyzések összesített tápértékei minden felhasználói módosítás után (legyen szó receptszerkesztésről vagy étkezés törléséről) valós időben, adatkonzisztencia hiba nélkül frissüljenek.

A kliensoldali fejlesztés hatékonyságát az NSwag kódgenerátor bevezetésével növeltem, amellyel automatizáltam a frontend és backend közötti típusbiztos kommunikációt. A felhasználói élményt reaktív űrlapokkal és a Chart.js könyvtárral megvalósított dinamikus adatvizualizációval javítottam. A rendszer stabilitását és az üzleti logika helyességét xUnit és InMemory Database segítségével futtatott automatizált integrációs tesztekkel igazoltam.

A jövőben a rendszert, vonalkód-leolvasóval és különböző bejelentkezési lehetőséggel tervezem tovább bővíteni. A projekt során szerzett legfontosabb szakmai tapasztalatom a „Full Stack” szemlélet elsajátítása volt: megtanultam, hogy a tiszta architektúra, a jól definiált API és az automatizált tesztelés nem csupán elméleti elvárások, hanem a hosszútávon fenntartható és hibatűrő szoftver elengedhetetlen alapfeltételei.

7 Irodalomjegyzék

- [1] [Online]. Available: https://en.wikipedia.org/wiki/Quantified_self.
- [2] „ASP:NET Core áttekintése,” [Online]. Available: <https://learn.microsoft.com/hu-hu/aspnet/core/overview?view=aspnetcore-9.0>.
- [3] „Jwtbearer,” [Online]. Available: <https://www.nuget.org/packages/Microsoft.AspNetCore.Authentication.JwtBearer>.
- [4] „Entity Framework documentation,” [Online]. Available: <https://learn.microsoft.com/hu-hu/ef/>.
- [5] „Email API Overview,” [Online]. Available: <https://dev.mailjet.com/email/guides/>.
- [6] „Nswag,” [Online]. Available: <https://github.com/RicoSuter/NSwag?tab=readme-ov-file>.
- [7] „Angular documentation,” [Online]. Available: <https://angular.dev/overview>.
- [8] „Typescript,” [Online]. Available: <https://www.typescriptlang.org/why-create-typescript/>.
- [9] [Online]. Available: <https://en.wikipedia.org/wiki/TypeScript>.
- [10] P. P. Benjámín, „Laboratóriumi információmenedzsment-rendszerek automatikus dokumentumgeneráló moduljának fejlesztése”.
- [11] [Online]. Available: <https://getbootstrap.com/>.
- [12] [Online]. Available: <https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver16>.
- [13] [Online]. Available: <https://code.visualstudio.com/>.
- [14] „Visual Studio,” [Online]. Available: <https://visualstudio.microsoft.com/>.

[15] [Online]. Available: <https://docs.github.com/en>.

[16] W3C, „HTML, The Web’s Core Language,” [Online]. Available: <http://www.w3.org/html/>. [Hozzáférés dátuma: 20 október 2015].

[17] „OpenAPI,” [Online]. Available: <https://swagger.io/resources/open-api/>.